

OWNER COMMAND REFERENCE

GOVERNED HUMAN CONTROL / EVIDENCE / RECOVERY

TS3-OCS-1.0 // VERSION 1.0 – FOUNDING EDITION

THIRSTY'S STANDARD v3

OWNER'S COCKPIT EXTENSION

End-to-End Standard for Human-Centered AI Command,
Control, Evidence, and Usability

DOCUMENT	TS3-OCS-1.0
CONTROL STATUS	Candidate Standard for Owner Ratification
PREPARED FOR	Jeremy Karrick / Thirsty's Projects LLC
SCOPE	Human-centered AI command, control, evidence, usability, security, resilience, and lifecycle
INHERITANCE	Additive extension to Thirsty's Standard v3; no v3 requirement is relaxed

No proof. No claim. No owner left outside the system.

*Made with care for the owner who built the engine before anyone remembered
to build the cockpit.*

Founding Statement

THE STANDARD IN ONE SENTENCE

A system is not truly owner-operable until its accountable owner can start it, understand it, direct it, verify it, recover it, and stop it without being forced to become its programmer.

The Owner's Cockpit exists because technical correctness and owner usability are not the same achievement. A repository may be carefully governed, comprehensively tested, reproducible, and deployment-ready while its owner still cannot simply load it, see what it does, and exercise real control. This extension closes that gap.

It treats the human operating layer as part of the system's architecture, not as a decorative frontend added after engineering. The cockpit carries owner intent through authority, policy, execution, evidence, recovery, and continuity. Every visible promise must be backed by runtime behavior. Every consequential control must reach a real enforcement boundary. Every claim must resolve to proof.

This document is written for owner-led AI systems, including solo-owner and small-team environments where the accountable person may coordinate AI agents and evaluate legitimacy without personally implementing every line of code. It rejects both unsafe autonomy and fictional bureaucracy. It preserves legitimate owner authority while refusing fabricated credentials, signatures, external evidence, or production claims.

FOUNDING PRINCIPLE

No proof. No claim. No owner left outside the system.

How to Use This Standard

Use this document as a requirements standard, design guide, engineering contract, acceptance framework, and audit reference. It is intended to be applied before architecture is frozen, during vertical-slice implementation, at owner acceptance, and throughout production operation.

1. Declare the cockpit scope, owner, environments, system boundaries, and target conformance level.
2. Map the owner jobs and complete journeys that the cockpit must support.
3. Adopt the normative requirements and assign each one an implementation owner and evidence method.
4. Build vertical slices that connect visible control to governed execution, evidence, recovery, and continuity.
5. Run the owner acceptance tests and maintain the requirements traceability matrix.
6. Hostile-review the result, close or disclose findings, and obtain legitimate owner ratification.
7. Re-run affected evidence whenever versions, policies, architecture, authority, data flows, or critical journeys change.

NORMATIVE LANGUAGE

SHALL and SHALL NOT are mandatory. SHOULD and SHOULD NOT express strong recommendations requiring a documented rationale when not followed. MAY identifies a permitted option. "Verified" is reserved for claims whose stated acceptance evidence has been obtained.

Authority, Inheritance, and Conflict Resolution

This document is an additive extension to Thirsty's Standard v3. It does not replace or weaken the prime directive, priority order, blocker rule, evidence rules, continuity requirements, production-readiness definition, hostile review, governance proof, fail-closed behavior, or final reporting obligations of v3.

Priority	Controlling principle
1	Safety and data protection
2	Owner's explicit instruction
3	Truthfulness and evidence
4	Current repository and system state
5	Minimal effective fix
6	Completeness within declared scope
7	Continuity preservation
8	Documentation and polish

When a cockpit requirement conflicts with a lower-level convenience, visual preference, delivery schedule, or engagement objective, the higher-priority rule controls. The interface must surface the conflict rather than silently resolving it against the owner.

Conformance Model

Level	Required outcome
OC-1 — Owner-Usable	The owner can launch, understand, use, verify a real workflow, recover from ordinary failures, and stop the system without developer-only knowledge.
OC-2 — Evidence-Governed	OC-1 plus governed action contracts, enforced authority, evidence receipts, durable continuity, no-bypass verification, and truthful operational state.
OC-3 — Production-Operational	OC-2 plus production identity, security, monitoring, incident response, backup/restore, deployment, rollback, accessibility, and owner-accepted operational gates.
OC-4 — Assured	OC-3 plus independent assurance, adversarial testing, calibrated risk evidence, stronger supply-chain and resilience proof, and periodic re-certification.

CONFORMANCE RULE

A higher level includes every mandatory requirement of the levels below it. Partial evidence may describe progress but may not close a mandatory criterion unless that criterion explicitly permits a substitute.

Owner-Centered Reference Architecture

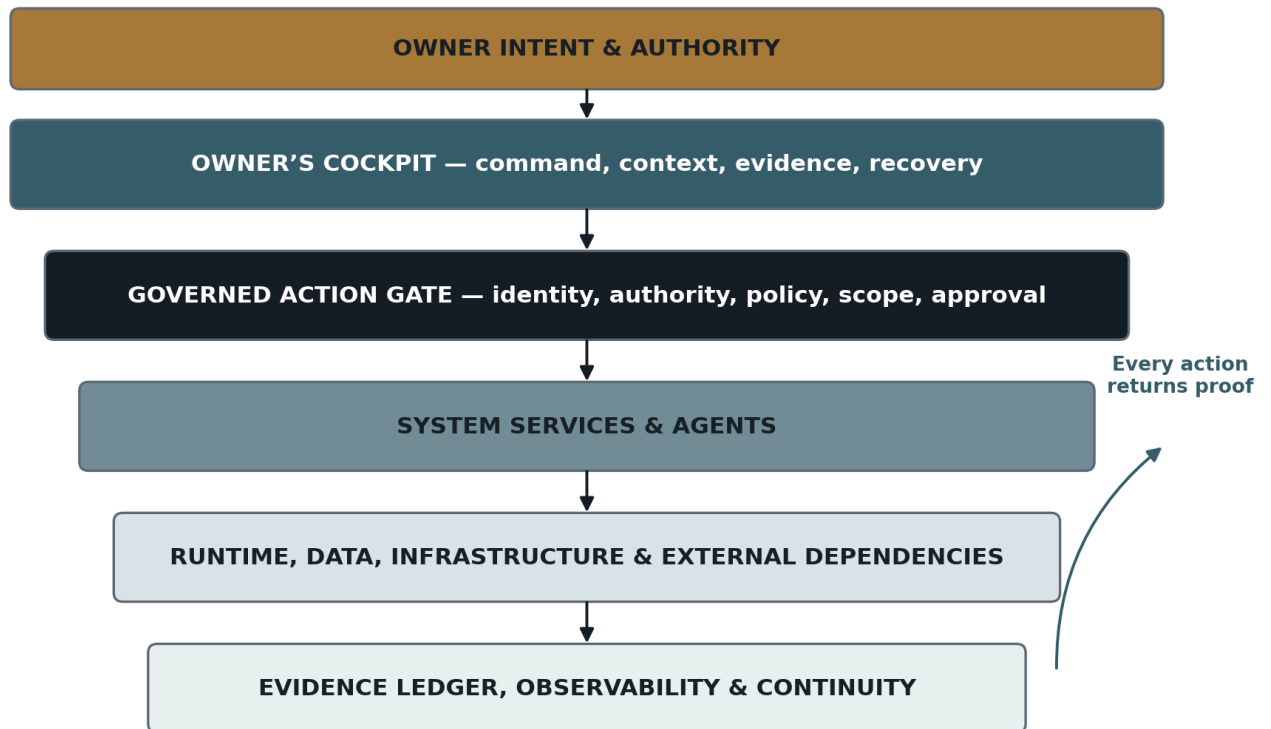
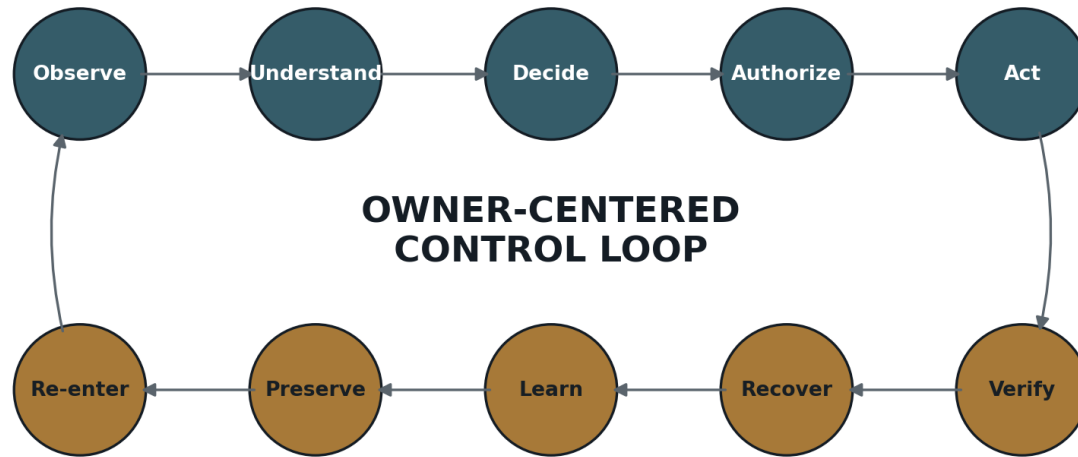


Figure 1. The cockpit is the governed bridge between owner authority and system execution. Evidence returns through the same operating loop.

The Complete Owner Control Loop



The cockpit is complete only when the owner can travel the full loop and return with evidence.

Figure 2. A complete cockpit supports the entire loop, including preservation, recovery, learning, and re-entry—not only action initiation.

Contents

PART I — FOUNDATION AND AUTHORITY

1. The Owner's Cockpit Mandate
2. Relationship to Thirsty's Standard v3
3. Owner, Operator, Developer, and User Roles
4. Authority and Precedence
5. Truthful Status and the No-Fake-Success Rule
6. Scope, Boundaries, and Completion

PART II — HUMAN FACTORS AND OWNER EXPERIENCE

7. Owner Jobs, Journeys, and Operational Intent
8. Cognitive Load Budget
9. Situational Awareness
10. Trust Calibration and Uncertainty
11. Decision Support Without Decision Seizure
12. Error Recovery Psychology and Owner Confidence

PART III — UI AND UX SYSTEM

13. Information Architecture and Navigation
14. Visual Hierarchy and Density
15. Controlled Status Language and Color Semantics
16. Interaction Grammar and Consequence Design
17. Forms, Configuration, and Safe Defaults
18. Accessibility and Inclusive Operation

PART IV — OPERATIONAL COCKPIT SURFACES

19. Home and Command Overview
20. Local Launch, Status, and Stop
21. Work, Task, and Action Control
22. Evidence, Receipts, and Verification
23. Incident, Degradation, and SAFE_HALT
24. Release, Deployment, and Rollback

PART V — AI AND MULTI-AGENT INTERACTION

25. Conversational Control and Grounded Dialogue
26. Agent Identity, Role, and Capability
27. Delegation and Bounded Autonomy
28. Multi-Agent Coordination and Handoffs
29. Independent Review and Adversarial Verification
30. Model Failure, Hallucination, and Fallback

PART VI — ENGINEERING ARCHITECTURE AND DATA CONTRACTS

31. Reference Architecture and Separation of Concerns

- 32. Action Contracts and State Machines
- 33. API, Event, and Evidence Schemas
- 34. Observability for Owner and Engineer
- 35. Configuration, Environment, and Secrets Architecture
- 36. Delivery Pipeline and Reproducibility

PART VII — SECURITY, SAFETY, ACCESSIBILITY, AND RESILIENCE

- 37. Identity, Authentication, and Session Safety
- 38. Authorization, Approval, and Separation of Duties
- 39. Data Protection and Privacy
- 40. Threat Modeling and Secure Interaction Design
- 41. Resilience, Offline Behavior, and Graceful Degradation
- 42. Backup, Restore, and Continuity of Control

PART VIII — LIFECYCLE, VERIFICATION, AND CONFORMANCE

- 43. Discovery and Owner Research
- 44. Design, Prototyping, and Architecture Alignment
- 45. Implementation and Definition of Done
- 46. Verification Strategy and Owner Acceptance
- 47. Release, Change, and Continuous Improvement
- 48. Conformance, Audit, and Ratification

Appendices A–Q — architecture, screens, status, contracts, alerts, tests, checklists, templates, traceability, anti-patterns, glossary, references, and ratification

TS3-OCS // COMMAND SECTOR 01 // OWNER CONTROLLED

PART I

FOUNDATION AND AUTHORITY

Defines why the Owner's Cockpit exists, who it serves, how it inherits Thirsty's Standard v3, and what may never be traded away for convenience.

ACCEPTANCE BOUNDARY

Complete only when owner outcomes are demonstrated through complete journeys and every mandatory requirement resolves to current evidence.

NO PROOF. NO CLAIM. NO OWNER LEFT OUTSIDE THE SYSTEM.

1. The Owner's Cockpit Mandate

OWNER OUTCOME

The owner can personally enter the system, understand its condition, direct real work, verify the result, and leave it in a known state.

The Owner's Cockpit is the governed human-system operating layer between owner intent and technical execution. It is not a decorative dashboard, a thin launcher, or a report viewer. It is the place where authority, context, action, evidence, recovery, and continuity become usable by the person accountable for the system.

Why this matters

A system may pass thousands of tests and still fail its owner if it cannot be started, understood, operated, or stopped without developer mediation. That failure is not cosmetic. It is a broken control pathway between ownership and operation.

Implementation standard

- Design from the owner's job-to-be-done rather than from the existing service topology.
- Make the shortest safe path visible: start, understand, act, verify, recover, stop.
- Treat owner usability as a production capability with tests and evidence, not as documentation polish.
- Keep the cockpit honest when underlying capabilities are partial, absent, degraded, or externally blocked.
- Provide a coherent operating surface while preserving component boundaries underneath.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-1.1	The system SHALL provide an owner-accessible entry point that does not require source-code editing or undocumented commands.	Recorded launch path, executable entry point, owner guide.	A non-programmer owner starts from a stopped machine and reaches the cockpit.	Open
OC-1.2	The cockpit SHALL expose current system state, available actions, and known limitations before asking the owner to act.	State snapshot, capability inventory, limitation registry.	First-session walkthrough identifies what is running and what is not.	Unknown
OC-1.3	The owner SHALL be able to complete at least one real governed workflow and observe its evidence trail.	Action receipt, backend trace, resulting state.	Execute the workflow twice from clean local starts.	Unproven

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?
- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

Accepted only when the owner—not merely an engineer or test runner—can complete the full start-to-proof-to-stop loop.

2. Relationship to Thirsty's Standard v3

OWNER OUTCOME

Every cockpit behavior extends the same truth, evidence, safety, continuity, and completion rules that govern the engineering beneath it.

This extension is additive. Thirsty's Standard v3 remains the governing execution contract; the Owner's Cockpit Standard translates those obligations into human-facing interaction, interface, operational, and acceptance requirements.

Why this matters

Without explicit inheritance, a polished interface can become a loophole: a place where uncertain state is beautified, partial work is called complete, or ungoverned actions escape the controls applied elsewhere.

Implementation standard

- Apply the v3 priority order to every interaction and design conflict.
- Map no-fake-success, evidence-before-claims, fail-closed, continuity, hostile review, and production readiness into visible UI behavior.
- Do not let a surface claim more certainty, readiness, authority, or safety than runtime evidence supports.
- Treat broken links, fake commands, dead buttons, and misleading labels as pathway-integrity defects.
- Use the v3 final-report discipline for cockpit implementation and releases.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-2.1	No cockpit claim SHALL exceed the verification state permitted by Thirsty's Standard v3.	Claim-to-evidence mapping and UI state tests.	Remove or falsify evidence and confirm the claim degrades to Unknown/Not verified.	False claim
OC-2.2	Every cockpit action path SHALL preserve the same governance and verification controls as direct backend execution.	Route map, policy-gate traces, negative tests.	Attempt the action through every UI/API path and verify identical enforcement.	Bypass
OC-2.3	Cockpit work SHALL maintain a truthful operational continuity record.	Current continuity map and change evidence.	A new operator reconstructs the last session without chat memory.	Stale

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?
- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

3. Owner, Operator, Developer, and User Roles

OWNER OUTCOME

The interface respects that the accountable owner may not be the person who writes the code—and does not punish that distinction.

The standard separates legal or operational accountability from technical implementation skill. One human may hold several roles, but the cockpit must not assume that ownership requires command-line fluency or that a developer automatically holds owner authority.

Why this matters

Role confusion causes both unsafe power and needless obstruction. A solo owner can be trapped behind fictional enterprise approvals, while a technically capable operator can be granted authority they were never given.

Implementation standard

- Model authority, capability, accountability, and technical skill as separate attributes.
- Permit a sole owner to hold legitimate development authority for repository-controlled work.
- Require explicit delegation when another actor may approve, deploy, publish, sign, or alter protected state.
- Never infer authority from UI access, technical expertise, employment title, or agent confidence.
- Show the acting identity, delegated scope, and consequence boundary before consequential actions.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-3.1	The cockpit SHALL distinguish owner, operator, developer, reviewer, and end-user capabilities in its authorization model.	Role matrix, policy tests, identity records.	Test each role against allowed and denied actions.	Overbroad
OC-3.2	A solo owner configuration SHALL NOT require invented organizational roles to complete repository-controlled development work.	Solo-owner policy profile and test fixture.	Complete a repository workflow without fictional CAB or team approvals.	Artificial block
OC-3.3	External authority requirements SHALL remain explicit and shall not be simulated by owner mode.	External-dependency registry and denial receipts.	Remove credentials or external assets and verify fail-closed behavior.	Fabricated

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?
- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

4. Authority and Precedence

OWNER OUTCOME

When instructions conflict, the owner can see which rule controls, why it controls, and what action remains safely possible.

The cockpit operationalizes the v3 priority order and makes conflict resolution observable. Safety and data protection lead; explicit owner instruction follows; truth and evidence constrain claims; current state controls feasibility; minimal effective completion is preferred.

Why this matters

Hidden precedence turns governance into surprise. If the system silently chooses a lower-priority goal over safety, evidence, or explicit instruction, the owner cannot meaningfully supervise it.

Implementation standard

- Store authority rules in machine-enforceable policy, not only prose.
- Display the controlling rule when an action is denied, deferred, narrowed, or escalated.
- Preserve safe partial progress when only part of a request is blocked.
- Require a reason and evidence for every policy override, exception, or emergency action.
- Version policy decisions and bind approvals to the evaluated policy version.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-4.1	Every consequential decision SHALL identify its controlling authority and policy version.	Decision receipt with actor, authority, policy hash.	Inspect a completed and denied action receipt.	Unattributed
OC-4.2	A lower-priority objective SHALL NOT override a higher-priority rule.	Conflict tests across priority pairs.	Inject conflicting instructions and verify deterministic resolution.	Priority violation
OC-4.3	When safe partial work remains, the cockpit SHALL present and preserve that path.	Partial-scope plan and evidence.	Block one dependency and confirm unaffected work remains executable.	Needless stop

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?

- Could stale, partial, cached, or simulated data be mistaken for current verified state?
- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

5. Truthful Status and the No-Fake-Success Rule

OWNER OUTCOME

The owner never has to guess whether “done” means executed, verified, partially completed, simulated, or merely planned.

The cockpit uses controlled status language tied to observable states. Created, modified, tested, verified, failed, blocked, pending, and not verified are distinct. Visual polish cannot convert one state into another.

Why this matters

Ambiguous green indicators and optimistic summaries are among the fastest ways to destroy operator trust. The owner must be able to tell what happened, what was checked, and what remains.

Implementation standard

- Define a single controlled status dictionary used by UI, APIs, agents, reports, and evidence records.
- Separate action completion from outcome verification and production readiness.
- Treat missing, stale, malformed, or contradictory telemetry as Unknown rather than healthy.
- Show partial completion by scope, not as an averaged success percentage.
- Never close a whole-gate criterion using partial substitutes unless the criterion explicitly permits it.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-5.1	Status labels SHALL have machine-readable definitions and permitted transitions.	Status schema and transition tests.	Attempt invalid transitions and verify rejection.	Invalid state
OC-5.2	Success SHALL require the exact acceptance evidence defined for that action or gate.	Acceptance contract and evidence receipt.	Withhold one required artifact and verify success is impossible.	False success
OC-5.3	Unknown, stale, partial, and simulated data SHALL be visibly distinguishable from verified current data.	Freshness metadata and visual regression tests.	Expire telemetry and inspect the rendered state.	Misleading

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?

- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?
- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

6. Scope, Boundaries, and Completion

OWNER OUTCOME

The owner sees what the system has agreed to do, what it did not agree to do, and what “complete” means before work begins.

Every cockpit task establishes a declared mode and bounded scope. The interface should prevent both underbuilding—scaffolds presented as systems—and endless expansion disguised as responsibility.

Why this matters

A cockpit that does not show scope creates two opposite failures: agents stop early because they can redefine completion downward, or they continue indefinitely because nothing defines enough.

Implementation standard

- Capture objective, in-scope work, preservation requirements, exclusions, acceptance criteria, and stop conditions.
- Show proposed scope expansion before execution and require owner authorization where material.
- Track blocked, pending, and external work separately from completed repository-controlled work.
- Use “Ready,” “Not ready,” “Ready except,” or “Blocked by” as completion outcomes.
- Require a final reconciliation between requested scope and delivered evidence.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-6.1	Every non-trivial action SHALL have an explicit scope contract before execution.	Scope record attached to action.	Inspect action history and reconstruct its boundaries.	Unbounded
OC-6.2	Material scope changes SHALL require a visible proposal and recorded authorization.	Change request and approval receipt.	Attempt silent expansion and verify it is denied or paused.	Scope drift
OC-6.3	Completion SHALL be reconciled against every acceptance criterion, not inferred from effort or elapsed time.	Criterion-by-criterion closure table.	Select a criterion and trace it to current evidence.	Unreconciled

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?
- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

TS3-OCS // COMMAND SECTOR 02 // OWNER CONTROLLED

PART II

HUMAN FACTORS AND OWNER EXPERIENCE

Defines how the cockpit protects attention, comprehension, confidence, and agency under ordinary work, failure, urgency, and uncertainty.

ACCEPTANCE BOUNDARY

Complete only when owner outcomes are demonstrated through complete journeys and every mandatory requirement resolves to current evidence.

NO PROOF. NO CLAIM. NO OWNER LEFT OUTSIDE THE SYSTEM.

7. Owner Jobs, Journeys, and Operational Intent

OWNER OUTCOME

The cockpit is organized around what the owner is trying to accomplish, not around internal package names or infrastructure trivia.

Owner-centered design starts with operational journeys: launch and explore, supervise work, review evidence, approve a consequence, recover from failure, release a candidate, and shut down safely. Each journey must be complete across UI, backend, evidence, and recovery.

Why this matters

Service-oriented navigation forces the owner to reverse-engineer architecture before performing basic work. That transfers system complexity to the wrong person and conceals missing end-to-end pathways.

Implementation standard

- Maintain an owner-job inventory with frequency, consequence, urgency, and evidence needs.
- Map each job from trigger through completion, recovery, and continuity update.
- Use owner language for primary navigation and technical language as optional detail.
- Test critical journeys from a stopped or degraded state, not only an already-prepared environment.
- Treat an incomplete journey as a product defect even when every isolated component passes.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-7.1	Every primary cockpit surface SHALL map to at least one documented owner job.	Job-to-surface matrix.	Select a surface and identify the owner outcome it enables.	Orphaned
OC-7.2	Each critical owner journey SHALL include entry, action, evidence, recovery, and exit states.	Journey map and end-to-end test.	Run from initial trigger to known final state.	Broken journey
OC-7.3	Navigation labels SHALL use owner-recognizable language before implementation-specific terminology.	Terminology test with owner.	Owner locates target task without coaching.	Developer-centric

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?
- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

8. Cognitive Load Budget

OWNER OUTCOME

The owner can identify what matters now without parsing every log, service, agent, and warning at once.

The cockpit treats attention as a scarce operational resource. It should summarize system condition, surface consequential change, and progressively reveal supporting detail without hiding proof.

Why this matters

Excess information is not transparency. Unranked telemetry, simultaneous alerts, and dense control panels increase error rates and can conceal the one fact requiring action.

Implementation standard

- Define a visual and informational budget for the default view.
- Prioritize by consequence, urgency, owner actionability, and evidence freshness.
- Use progressive disclosure: summary, reason, evidence, raw detail.
- Avoid duplicate notifications for the same underlying incident.
- Preserve context when the owner moves between summary and detail.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-8.1	The default cockpit view SHALL present a bounded set of owner-relevant conditions and next actions.	Information inventory and rendered review.	Owner identifies top condition and next action within one minute.	Overloaded
OC-8.2	Technical detail SHALL remain reachable without dominating the first decision layer.	Progressive-disclosure interaction test.	Move from summary to raw evidence and back without losing context.	Hidden or noisy
OC-8.3	Duplicate symptoms from one cause SHALL be grouped under a correlated incident when evidence permits.	Correlation record and alert test.	Trigger one shared failure and inspect notification grouping.	Alert flood

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?

- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

9. Situational Awareness

OWNER OUTCOME

At any moment the owner can answer: What is happening? Why? What changed? What can I do? What will happen next?

Situational awareness combines current state, recent change, causal context, projection, and available control. The cockpit should support perception, comprehension, and anticipation without overstating prediction.

Why this matters

A static dashboard can show many numbers while leaving the owner operationally blind. State without change history, cause, or consequence is insufficient for supervision.

Implementation standard

- Show current mode, environment, branch/version, active work, degraded dependencies, and pending authority.
- Provide a recent-change timeline connected to actions and evidence.
- Distinguish measured facts from system interpretations and forecasts.
- Expose the likely consequence of inaction where it is grounded and material.
- Keep time, timezone, freshness, and environment visible for operational data.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-9.1	The cockpit SHALL identify the active environment, version, mode, and actor on every consequential surface.	Rendered surfaces and context metadata.	Switch environment or actor and verify prominent change.	Context loss
OC-9.2	Current conditions SHALL be linked to relevant changes, actions, and evidence.	Timeline correlation identifiers.	Trace a displayed condition to its originating event.	Untraceable
OC-9.3	Predictions and interpretations SHALL be labeled separately from observed facts.	Provenance labels and schema.	Inspect mixed fact/forecast presentation.	Conflated

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?

- Could stale, partial, cached, or simulated data be mistaken for current verified state?
- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

10. Trust Calibration and Uncertainty

OWNER OUTCOME

The owner neither blindly trusts the system nor dismisses it; confidence matches the quality and limits of the evidence.

The cockpit should communicate uncertainty, provenance, data age, coverage, and known blind spots. Confidence values must describe a defined quantity and must never serve as decorative reassurance.

Why this matters

Overconfident interfaces invite automation bias. Vague caveats invite learned helplessness. Calibrated trust requires specific, actionable boundaries around what is known and unknown.

Implementation standard

- Use uncertainty language that names the missing fact or weak evidence.
- Show provenance and method for high-consequence recommendations.
- Never use a confidence percentage without a defined interpretation and validation basis.
- Allow the owner to request supporting evidence, alternatives, and dissenting signals.
- Track prediction or recommendation outcomes so calibration can improve.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-10.1	High-consequence recommendations SHALL disclose evidence basis, uncertainty, and material limitations.	Recommendation record and UI inspection.	Review a recommendation with missing data and verify explicit limits.	Overstated
OC-10.2	Confidence indicators SHALL have documented semantics and calibration evidence.	Metric definition and validation report.	Ask two operators to interpret the indicator consistently.	Meaningless
OC-10.3	The owner SHALL be able to inspect evidence and alternative options before authorization.	Interaction path and decision receipt.	Open evidence and compare alternatives without leaving the decision context.	Coercive

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?

- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

11. Decision Support Without Decision Seizure

OWNER OUTCOME

The system helps the owner understand tradeoffs and consequences while preserving the owner's legitimate final authority.

Decision support should structure options, evidence, reversibility, risk, and expected consequence. It must not silently convert a recommendation into an action or manipulate urgency to force acceptance.

Why this matters

An AI system can appear helpful while narrowing the owner's choices, hiding dissent, or preselecting irreversible actions. That is control disguised as assistance.

Implementation standard

- Separate recommendation, authorization, execution, and verification as distinct events.
- Present the recommended option alongside meaningful alternatives and “do nothing” where valid.
- Identify reversible versus irreversible consequences before authorization.
- Avoid dark patterns, forced defaults, countdown pressure, and misleading button hierarchy.
- Record the owner's decision and the evidence available at that time.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-11.1	A recommendation SHALL NOT execute without the authority required by the action contract.	Separate recommendation and execution records.	Generate a recommendation and confirm no side effect occurs.	Seized
OC-11.2	Material alternatives and consequences SHALL be visible before approval.	Decision comparison and usability test.	Owner explains tradeoffs before committing.	Obscured
OC-11.3	Irreversible or high-impact actions SHALL require explicit, unambiguous confirmation bound to exact scope.	Confirmation receipt and scope hash.	Modify scope after confirmation and verify re-approval is required.	Stale approval

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?

- Could stale, partial, cached, or simulated data be mistaken for current verified state?
- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

12. Error Recovery Psychology and Owner Confidence

OWNER OUTCOME

When something fails, the owner receives a truthful path forward instead of blame, panic, or a dead end.

Failure interactions should preserve orientation, completed work, evidence, and safe options. The system must explain what failed in owner language, what remains trustworthy, and how to retry, repair, roll back, or escalate.

Why this matters

Cryptic errors and destructive retry loops teach owners not to touch their own systems. Confidence is rebuilt by predictable recovery, not by hiding failure.

Implementation standard

- State the failed operation, affected scope, preserved state, and next safe action.
- Keep technical diagnostics available but separate from owner-facing recovery guidance.
- Prevent duplicate side effects during retry through idempotency or explicit reconciliation.
- Offer rollback only when a tested rollback path exists.
- Never blame the owner for system complexity or present missing developer knowledge as user error.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-12.1	Every actionable failure SHALL provide a safe next step or an explicit real blocker.	Error catalog and recovery path.	Induce representative failures and follow the offered path.	Dead end
OC-12.2	Retry SHALL NOT duplicate consequential side effects.	Idempotency key, reconciliation log, tests.	Interrupt and retry an operation; inspect resulting state.	Duplicate
OC-12.3	Recovery guidance SHALL identify what remains safe and verified after failure.	Post-failure state report.	Owner distinguishes preserved and uncertain state.	Disorienting

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?

- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

TS3-OCS // COMMAND SECTOR 03 // OWNER CONTROLLED

PART III

UI AND UX SYSTEM

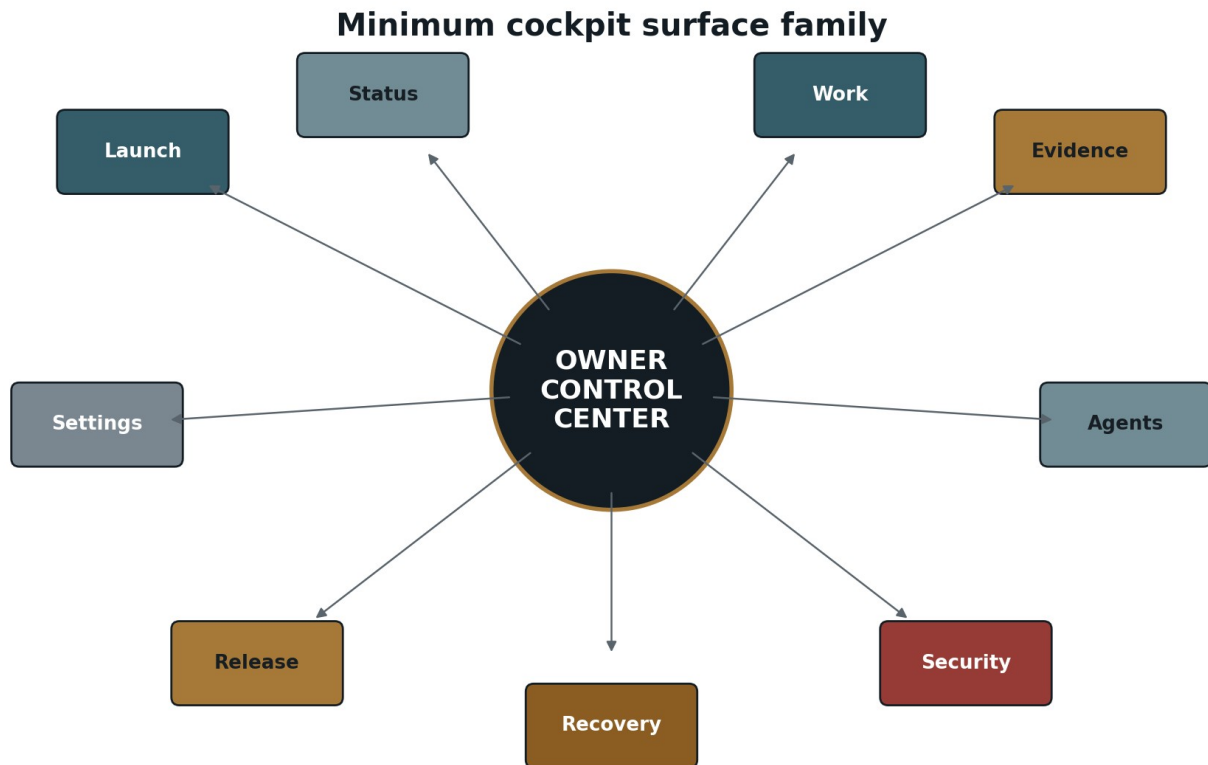
Defines the visible language, interaction grammar, navigation, accessibility, and evidence presentation that make the cockpit coherent and trustworthy.

ACCEPTANCE BOUNDARY

Complete only when owner outcomes are demonstrated through complete journeys and every mandatory requirement resolves to current evidence.

NO PROOF. NO CLAIM. NO OWNER LEFT OUTSIDE THE SYSTEM.

COMMAND MAP // SECTOR 03



The visible system is organized around owner control, not internal package topology.

13. Information Architecture and Navigation

OWNER OUTCOME

The owner knows where they are, where to go next, and how to return without losing work or context.

The cockpit uses a stable task-oriented information architecture. Primary destinations should represent owner concerns such as Home, Work, Agents, Evidence, Release, Security, Recovery, and Settings rather than a mirror of package topology.

Why this matters

Navigation is a control system. Inconsistent names, hidden modes, and context-destroying transitions can produce operational mistakes even when individual screens are well designed.

Implementation standard

- Maintain a canonical route and surface inventory.
- Provide persistent environment, mode, identity, and system-condition context.
- Preserve filters, selected entity, and return location across drill-downs.
- Support direct links to durable objects such as actions, incidents, evidence, and releases.
- Avoid duplicate destinations that expose different truth for the same underlying object.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-13.1	Primary navigation SHALL be stable, owner-task-oriented, and documented.	Route inventory and navigation tests.	Owner locates each critical task from Home.	Lost
OC-13.2	Deep links SHALL restore the referenced object, environment, and safe context.	URL/route tests.	Open a copied evidence link in a new session.	Contextless
OC-13.3	Multiple views of one object SHALL use a shared authoritative state source.	Data-contract map and consistency test.	Compare status across all views during an update.	Contradictory

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?

- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

14. Visual Hierarchy and Density

OWNER OUTCOME

The most consequential condition and next action are visually clear without drowning out evidence or nuance.

Visual hierarchy should encode importance, urgency, relationship, and actionability. Size, position, contrast, spacing, grouping, and typography are semantic tools, not decoration.

Why this matters

When every card, badge, and alert competes equally, the owner must perform the prioritization the system should have performed. Conversely, oversized reassurance can hide unresolved risk.

Implementation standard

- Reserve the strongest visual emphasis for conditions requiring owner attention.
- Use whitespace and grouping to reveal relationships and reduce scanning effort.
- Keep sustained prose readable and avoid shrinking type to fit more telemetry.
- Design dense expert views separately from the default owner overview.
- Test at realistic data volume, long names, error states, and zoom levels.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-14.1	Visual prominence SHALL correspond to defined consequence and actionability rules.	Design-token mapping and screenshot review.	Compare high and low severity conditions.	Misprioritized
OC-14.2	Critical surfaces SHALL remain usable at 200% zoom and common narrow widths.	Responsive and zoom test evidence.	Complete the primary task at 200% browser zoom.	Reflow failure
OC-14.3	Production-like data volume SHALL not cause clipping, overlap, or unreadable compression.	Visual regression fixtures.	Render maximum supported labels and records.	Overflow

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?

- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

15. Controlled Status Language and Color Semantics

OWNER OUTCOME

The owner can understand status without memorizing colors, guessing at synonyms, or confusing progress with proof.

Status communication combines controlled words, icons, structure, and optional color. Color reinforces meaning but never carries meaning alone. The same label means the same state everywhere.

Why this matters

Green can be dangerously vague: healthy service, passed test, completed task, approved release, and verified outcome are not interchangeable. A status system must encode exact state.

Implementation standard

- Use the canonical status dictionary from Appendix C.
- Pair color with text and, where useful, shape or iconography.
- Reserve red for failed, denied, or urgent unsafe conditions; amber for attention, risk, or waiting; green for verified success only.
- Do not use “active,” “ready,” “complete,” or “healthy” without a defined subject and criterion.
- Include freshness and scope near operational status.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-15.1	Status color SHALL never be the sole carrier of meaning.	Accessibility inspection and automated tests.	Use grayscale/color-blind simulation and identify every state.	Color-only
OC-15.2	Each status label SHALL name or imply its subject, scope, and verification meaning.	Status glossary and component tests.	Ask what is ready and against which criterion.	Ambiguous
OC-15.3	Verified-success styling SHALL be unavailable until required evidence is present.	Component state guard and test.	Attempt to render success without evidence.	Decorative success

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?

- Could stale, partial, cached, or simulated data be mistaken for current verified state?
- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

16. Interaction Grammar and Consequence Design

OWNER OUTCOME

Similar actions behave similarly, and dangerous actions feel meaningfully different before anything irreversible happens.

The cockpit defines a consistent interaction grammar for inspect, configure, preview, authorize, execute, pause, cancel, retry, rollback, and acknowledge. Consequence should be visible in placement, copy, confirmation, and evidence requirements.

Why this matters

Inconsistent action semantics train the owner to click by habit. That becomes hazardous when the same visual pattern sometimes previews and sometimes executes.

Implementation standard

- Use stable verbs with one meaning across the system.
- Separate navigation controls from state-changing controls.
- Preview scope and effects before consequential execution.
- Require stronger friction only as consequence and irreversibility rise.
- Return a durable result object after every state-changing action.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-16.1	Action verbs SHALL have a canonical semantic definition and consistent effect.	Interaction dictionary and cross-surface audit.	Compare every occurrence of a high-risk verb.	Inconsistent
OC-16.2	Consequential actions SHALL display target, scope, effect, reversibility, and authority before execution.	Confirmation surface and receipt.	Owner accurately predicts the result before confirming.	Unclear consequence
OC-16.3	Every state-changing action SHALL produce an inspectable result object.	Action ID, receipt, and history entry.	Refresh or reopen and locate the action result.	Ephemeral

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?

- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

17. Forms, Configuration, and Safe Defaults

OWNER OUTCOME

The owner can configure the system without knowing hidden formats, creating invalid combinations, or accidentally exposing production assets.

Forms should translate configuration contracts into understandable choices, validate early, explain consequences, and preserve recovery. Defaults must be safe for the declared mode and environment.

Why this matters

Configuration interfaces are often where usability and security fail together. A technically valid field can still invite dangerous entry, while silent defaults can conceal material behavior.

Implementation standard

- Generate form constraints from authoritative schemas where possible.
- Use examples, units, allowed ranges, and dependency guidance near the field.
- Validate before execution and again at the backend boundary.
- Distinguish unset, inherited, defaulted, user-supplied, and secret values.
- Provide diff, export, restore, and reset paths for material configuration.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-17.1	Configuration inputs SHALL be validated at both interface and execution boundaries.	Shared schema and negative tests.	Bypass client validation and verify backend rejection.	Invalid accepted
OC-17.2	Defaults SHALL be explicitly documented and safe for the active mode.	Default registry and mode tests.	Start with no local config and inspect resulting behavior.	Unsafe default
OC-17.3	Secret values SHALL never be echoed in plain text, logs, receipts, or exports.	Redaction tests and log scan.	Submit a canary secret and search all outputs.	Secret leak

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?

- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

18. Accessibility and Inclusive Operation

OWNER OUTCOME

The cockpit remains operable by keyboard, screen reader, zoom, alternative input, and varied sensory or cognitive needs across the complete owner process.

Accessibility is a pathway property. The target is WCAG 2.2 Level AA for the complete owner journeys, supplemented by WAI-ARIA Authoring Practices and tested human use.

Conformance of isolated components is not enough if the process breaks.

Why this matters

An inaccessible approval, recovery, authentication, or evidence step can exclude the accountable owner from control of their own system. That is an authority failure as well as a usability failure.

Implementation standard

- Use semantic HTML and native controls before custom widgets.
- Provide keyboard-visible focus, logical order, accessible names, status announcements, and error association.
- Meet contrast, reflow, target-size, timing, and authentication requirements for relevant content.
- Avoid motion, flashing, or auto-updating behavior that cannot be controlled.
- Test complete owner journeys with automated scans and manual assistive-technology checks.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-18.1	All critical owner journeys SHALL conform to WCAG 2.2 Level AA except documented, approved exceptions with remediation plans.	Automated reports, manual checklist, exception record.	Run the complete journey by keyboard and screen reader.	Nonconformant
OC-18.2	Dynamic status and errors SHALL be programmatically conveyed without stealing focus unexpectedly.	ARIA/live-region tests and manual review.	Trigger status updates using assistive technology.	Silent
OC-18.3	No critical action SHALL depend solely on pointer precision, color perception, hearing, or time-limited response.	Alternative-input and sensory review.	Complete actions using keyboard and non-color cues.	Exclusion

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?
- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

TS3-OCS // COMMAND SECTOR 04 // OWNER CONTROLLED

PART IV

OPERATIONAL COCKPIT SURFACES

Defines the minimum family of owner-facing surfaces required to launch, supervise, prove, secure, recover, release, and stop a real system.

ACCEPTANCE BOUNDARY

Complete only when owner outcomes are demonstrated through complete journeys and every mandatory requirement resolves to current evidence.

NO PROOF. NO CLAIM. NO OWNER LEFT OUTSIDE THE SYSTEM.

19. Home and Command Overview

OWNER OUTCOME

On entry, the owner immediately understands the system's condition, active work, unresolved attention, and safest next action.

Home is the orientation surface, not a collage of metrics. It should answer what is running, what changed, what needs attention, what is blocked, and what the owner can do now.

Why this matters

A home screen that optimizes for activity or visual richness can conceal operational truth. Its job is to establish a reliable mental model and direct the owner toward complete journeys.

Implementation standard

- Show environment, version, mode, actor, overall condition, active actions, and unresolved incidents.
- Display the last verified update time and any telemetry gaps.
- Prioritize owner-actionable conditions over passive statistics.
- Provide direct routes to evidence and recovery for every material condition.
- Avoid composite health scores unless their composition and limits are inspectable.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-19.1	Home SHALL answer the five orientation questions within one screen: where, what state, what changed, what needs action, and where proof lives.	Owner walkthrough and screen capture.	Owner answers all five without opening developer tools.	Disorienting
OC-19.2	Every material Home condition SHALL link to its evidence and applicable control.	Link tests and object IDs.	Open condition, proof, and next action.	Non-actionable
OC-19.3	Telemetry gaps SHALL reduce confidence visibly rather than disappear from summary.	Missing-data simulation and screenshot.	Disable one source and inspect Home.	False healthy

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?

- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

20. Local Launch, Status, and Stop

OWNER OUTCOME

The owner can start the complete local system, open the correct interface, verify readiness, and stop it safely with obvious commands or controls.

Local owner mode is a first-class operating mode. It must not require production DNS, signing, publication, external monitoring, or unrelated credentials. It must still preserve governance and truthful capability boundaries.

Why this matters

Healthy components are not enough if the owner cannot find the URL, identify startup completion, or safely shut down. Launch is an end-to-end product journey.

Implementation standard

- Provide one obvious start path, one status path, and one stop path.
- Preflight dependencies, ports, storage, configuration, and local permissions before startup.
- Print or display the exact URL and expected startup duration.
- Confirm required services and a real owner-visible workflow, not only container liveness.
- Make shutdown graceful, bounded, and verifiable; preserve diagnostics on failure.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-20.1	A documented single owner action SHALL start all required local components or explain the exact real blocker.	Launcher, preflight log, owner guide.	Start from all components stopped.	Unlaunchable
OC-20.2	Local readiness SHALL include a successful governed end-to-end owner workflow.	Workflow receipt and service traces.	Perform the declared local acceptance action twice.	Liveness only
OC-20.3	Stop SHALL terminate or quiesce all owned local components and report residual state.	Shutdown log and process/container inventory.	Stop, then verify no unintended owned process remains.	Residual unknown

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?

- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

21. Work, Task, and Action Control

OWNER OUTCOME

The owner can create, inspect, prioritize, authorize, pause, cancel, and verify work without losing scope or provenance.

The work surface represents durable actions rather than chat turns. Each action carries objective, scope, actor, authority, state, dependencies, evidence, and recovery information.

Why this matters

Conversation alone is a weak operating ledger. Messages can become separated from execution, disappear from context, or imply work that never became a governed action.

Implementation standard

- Create a stable action ID before consequential execution.
- Show plan, current step, dependencies, elapsed time, and expected next transition.
- Distinguish queued, awaiting authority, executing, paused, blocked, failed, verified, and cancelled.
- Allow safe pause or cancellation only where semantics are defined.
- Preserve action history and result evidence beyond the live session.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-21.1	Every governed work item SHALL have a durable identity and state history.	Action record and event stream.	Refresh/restart and recover the action.	Session-only
OC-21.2	Pause, cancel, and retry SHALL have explicit backend semantics and tested state transitions.	State machine and interruption tests.	Invoke each control during representative work.	Cosmetic control
OC-21.3	The owner SHALL see the exact scope and authority currently bound to an action.	Action detail and signed/hashed scope.	Compare presented scope to execution trace.	Scope mismatch

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?

- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

22. Evidence, Receipts, and Verification

OWNER OUTCOME

The owner can inspect the proof behind any meaningful claim without searching across terminals, chats, and disconnected reports.

Evidence is a first-class object. The cockpit should issue action receipts, verification records, provenance, artifact links, integrity metadata, and a clear distinction between executed and recommended checks.

Why this matters

Evidence hidden in logs cannot reliably govern claims. Conversely, a polished receipt without raw supporting artifacts can become safety theater.

Implementation standard

- Represent evidence with subject, claim, scope, method, environment, time, actor/verifier, result, artifacts, and integrity.
- Bind receipts to immutable or content-addressed identifiers where practical.
- Show evidence freshness, coverage, and supersession.
- Permit drill-down from claim to raw artifact while preserving owner-readable summary.
- Prevent deletion or silent mutation of material audit evidence.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-22.1	Every technical success claim SHALL reference a current evidence record.	Claim ID linked to evidence ID.	Select a claim and resolve every referenced artifact.	Unsupported
OC-22.2	Evidence records SHALL disclose scope, environment, version, and verification method.	Receipt schema validation.	Compare evidence from two environments.	Context-free
OC-22.3	Material evidence SHALL be append-only or tamper-evident, with corrections recorded as new events.	Hash/signature chain or immutable store tests.	Attempt mutation and inspect detection/audit.	Mutable history

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?

- Could stale, partial, cached, or simulated data be mistaken for current verified state?
- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

23. Incident, Degradation, and SAFE_HALT

OWNER OUTCOME

The owner can recognize unsafe or degraded conditions, contain them, preserve evidence, and recover without improvisation.

Incident surfaces combine detection, consequence, affected scope, containment, communication, evidence, and recovery. SAFE_HALT is an enforced state that prevents prohibited continuation while preserving diagnostic access and controlled recovery.

Why this matters

A red banner is not incident control. Without enforced consequence, ownership, and recovery state, alerts merely describe risk while the system continues creating it.

Implementation standard

- Define incident severity by consequence and required response, not emotion or volume.
- Provide acknowledge, assign, contain, investigate, recover, and close transitions.
- Implement SAFE_HALT at the execution boundary and expose its cause and allowed operations.
- Preserve logs, evidence, and last-known-good references during containment.
- Require verification before returning from halt or degraded mode.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-23.1	SAFE_HALT SHALL prevent prohibited execution through every path while retaining authorized diagnostic and recovery access.	Denial tests across UI/API/agent routes.	Enter SAFE_HALT and attempt all consequential paths.	Bypass
OC-23.2	Incident closure SHALL require containment and recovery evidence, not acknowledgement alone.	Closure criteria and incident receipt.	Attempt to close before verification.	Premature closure
OC-23.3	The cockpit SHALL preserve the condition, timeline, evidence, and actions associated with an incident.	Incident bundle and export test.	Reconstruct the incident after restart.	Lost evidence

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?

- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?
- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

24. Release, Deployment, and Rollback

OWNER OUTCOME

The owner can see exactly what candidate is being released, which gates passed, what remains external, and how to stop or reverse the change.

The release surface binds immutable candidate identity to gate evidence, approvals, environment, deployment execution, observation, and rollback readiness. Local success and production authorization remain distinct.

Why this matters

Release ambiguity is dangerous because “the code passed” can refer to a different commit, environment, or partial gate. The cockpit must prevent evidence drift and substitute closure.

Implementation standard

- Identify the release candidate by immutable commit and artifact digest.
- Show each gate’s exact success criterion, current evidence, and closure authority.
- Separate repository-controlled readiness from owner, infrastructure, and external obligations.
- Require deployment and rollback plans that reference real commands, assets, and decision thresholds.
- Observe post-deployment behavior and record the production verdict.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-24.1	Release evidence SHALL be bound to the exact candidate and environment it qualifies.	Commit/artifact digest in each gate receipt.	Change candidate and verify evidence invalidation.	Evidence drift
OC-24.2	A whole-gate criterion SHALL remain open until its defined end-to-end success condition is met.	Gate contract and execution receipt.	Pass individual steps but withhold whole-gate result.	Substitute closure
OC-24.3	Rollback readiness SHALL be tested or truthfully labeled unverified before deployment.	Rollback rehearsal or explicit pending status.	Exercise in staging or inspect truthful block.	Untested claim

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?

- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

TS3-OCS // COMMAND SECTOR 05 // OWNER CONTROLLED

PART V

AI AND MULTI-AGENT INTERACTION

Defines how intelligent assistants, specialist agents, and autonomous workflows become visible, bounded, governable, and useful to the owner.

ACCEPTANCE BOUNDARY

Complete only when owner outcomes are demonstrated through complete journeys and every mandatory requirement resolves to current evidence.

NO PROOF. NO CLAIM. NO OWNER LEFT OUTSIDE THE SYSTEM.

25. Conversational Control and Grounded Dialogue

OWNER OUTCOME

Conversation helps the owner understand and direct the system, but never becomes an untracked bypass around governed action.

The conversational layer should answer questions, explain state, prepare plans, and propose actions using current cockpit data. Any state-changing request must become a durable governed action with scope, authority, and evidence.

Why this matters

Natural language can blur the line between discussion and execution. Without explicit conversion, the owner may believe an operation occurred when only text was produced—or an operation may occur without clear authorization.

Implementation standard

- Label whether a response is explanation, recommendation, plan, simulation, action request, or executed result.
- Ground operational answers in current state and cite cockpit objects or evidence.
- Convert consequential instructions into reviewable action contracts before execution.
- Preserve conversation-to-action linkage and resulting evidence.
- Allow the owner to correct misunderstood intent before consequence.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-25.1	Conversational text SHALL NOT be presented as executed work without a linked successful action receipt.	Message type and action linkage.	Ask for a change and inspect state before authorization.	Text-as-action
OC-25.2	Operational answers SHALL identify their data sources and freshness.	Source references and timestamps.	Disconnect a source and query current state.	Ungrounded
OC-25.3	Materially ambiguous requests SHALL be clarified or narrowed before execution.	Ambiguity policy and interaction tests.	Issue an underspecified destructive request.	Assumed intent

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?

- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?
- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

26. Agent Identity, Role, and Capability

OWNER OUTCOME

The owner knows which agent is acting, what it is allowed to do, what tools it can reach, and where its authority ends.

Each agent is a governed actor with identity, role contract, capability set, authority scope, model/runtime provenance, and current assignment. Friendly names do not replace verifiable identities.

Why this matters

In multi-agent systems, unlabelled handoffs and shared credentials make accountability collapse. The owner must distinguish who proposed, who reviewed, who executed, and who verified.

Implementation standard

- Maintain an agent registry with stable IDs and versioned role contracts.
- Show assigned objective, tools, data access, authority, and active constraints.
- Keep model capability separate from granted permission.
- Record agent-to-agent delegation and retained responsibility.
- Disable or quarantine agents whose identity, contract, or runtime cannot be verified.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-26.1	Every agent action SHALL be attributable to a stable agent identity and role-contract version.	Agent ID and contract hash in receipt.	Trace an action through handoffs.	Anonymous
OC-26.2	Agent capability SHALL NOT imply authority; actions require explicit policy grants.	Capability/permission matrix and denial tests.	Give a capable agent no grant and attempt action.	Capability bypass
OC-26.3	Unverified or drifted agent configurations SHALL be prevented from consequential execution.	Attestation/version check and quarantine event.	Alter an agent contract and attempt work.	Configuration drift

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?

- Could stale, partial, cached, or simulated data be mistaken for current verified state?
- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

27. Delegation and Bounded Autonomy

OWNER OUTCOME

The owner can delegate meaningful work without surrendering control over scope, consequence, or completion criteria.

Delegation contracts define objective, preservation requirements, authority, resource limits, duration, stop conditions, escalation thresholds, and evidence expectations. Autonomy is bounded by these terms, not by agent confidence.

Why this matters

Vague delegation causes either paralysis or overreach. “Finish everything” is only safe when the system can determine the current scope, preserve constraints, and stop at real boundaries.

Implementation standard

- Make delegation inspectable before and during execution.
- Support time, cost, tool, data, and consequence budgets.
- Require reauthorization when material scope or consequence changes.
- Allow the owner to pause, narrow, or revoke delegation.
- Preserve completed safe work when a delegated subtask becomes blocked.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-27.1	Autonomous work SHALL remain within a recorded delegation contract.	Delegation ID on every child action.	Attempt a child action outside scope.	Overreach
OC-27.2	Material delegation changes SHALL require explicit owner authorization.	Change proposal and approval record.	Expand resource or consequence boundary.	Silent expansion
OC-27.3	Revocation SHALL prevent new consequential actions and define handling of in-flight work.	Revocation event and execution tests.	Revoke during operation and inspect state.	Unstoppable

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?

- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

28. Multi-Agent Coordination and Handoffs

OWNER OUTCOME

The owner sees how specialists divide work, where responsibility moves, and whether cross-agent conclusions agree.

Coordination should create a shared work graph with explicit dependencies, handoff artifacts, acceptance criteria, and responsibility. Agents should not rely on hidden conversational memory to transmit critical state.

Why this matters

Parallel agents can multiply output while fragmenting truth. Without durable handoffs, one agent may verify a different version, repeat work, or inherit an unsupported claim.

Implementation standard

- Represent multi-agent work as a dependency graph with stable action IDs.
- Require handoff packages containing scope, state, evidence, risks, and next action.
- Bind downstream work to exact upstream artifact versions.
- Surface conflicts, duplicate work, and contradictory evidence.
- Preserve one authoritative status source per work item.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-28.1	Every agent handoff SHALL include a durable, versioned handoff record.	Handoff artifact and receiving acknowledgement.	Restart the receiver and continue from record alone.	Chat-dependent
OC-28.2	Downstream verification SHALL identify the exact upstream artifact or state inspected.	Artifact digest and dependency link.	Change upstream output and verify invalidation.	Version mismatch
OC-28.3	Conflicting agent conclusions SHALL be surfaced and reconciled rather than silently averaged.	Conflict object and resolution receipt.	Inject contradictory assessments.	Hidden conflict

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?

- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

29. Independent Review and Adversarial Verification

OWNER OUTCOME

The owner can ask a separate agent or process to challenge work without that reviewer inheriting the original agent's assumptions.

Independent review separates creator and verifier roles for high-consequence work. The reviewer receives the objective, evidence, and artifact but must reconstruct critical claims rather than accepting summaries.

Why this matters

Self-review is necessary but not sufficient. Agents can preserve their own framing errors, overlook omitted scope, or rationalize an unsupported completion claim.

Implementation standard

- Use independent review where consequence, novelty, uncertainty, or irreversibility warrants it.
- Provide reviewers access to raw evidence and current artifact state.
- Require negative tests and bypass attempts, not only happy-path confirmation.
- Record disagreements, dispositions, and unresolved residual risk.
- Prevent the creator from silently editing the reviewed evidence after approval.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-29.1	High-consequence work SHALL receive independent verification under a defined review contract.	Reviewer identity, scope, and report.	Trace approval to independent evidence.	Self-certified
OC-29.2	Reviewers SHALL test denial, failure, and bypass paths relevant to the claim.	Negative-test record.	Inspect review coverage beyond happy path.	Shallow review
OC-29.3	Post-review artifact changes SHALL invalidate or narrow the prior approval.	Digest binding and invalidation event.	Modify reviewed artifact and inspect status.	Stale approval

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?

- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

30. Model Failure, Hallucination, and Fallback

OWNER OUTCOME

When an AI model is uncertain, unavailable, or wrong, the cockpit fails safely and preserves a usable non-deceptive path.

Models are untrusted components whose output may assist but cannot substitute for runtime proof, authority validation, or deterministic safety controls. The cockpit should detect grounding failures, unsupported claims, and degraded providers.

Why this matters

A fluent answer can look more reliable than a visible error. The system must prefer an honest unknown, deterministic fallback, or escalation over persuasive fabrication.

Implementation standard

- Validate model-produced action parameters against schemas and policy.
- Require source grounding for operational claims and detect unresolved references.
- Provide deterministic or manual fallback for critical owner controls.
- Show provider/model degradation and its effect on capability.
- Log model input/output provenance according to privacy and security rules.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-30.1	Model output SHALL NOT directly bypass deterministic validation, authority, or execution gates.	Architecture trace and malformed-output tests.	Inject invalid model parameters.	Model bypass
OC-30.2	Unsupported operational claims SHALL be marked unknown or rejected.	Grounding validator and test set.	Request state for nonexistent evidence.	Hallucinated fact
OC-30.3	Critical stop, status, and recovery controls SHALL remain available during model/provider outage.	Fallback path and outage test.	Disable all model services and operate controls.	Model dependency

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?

- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

TS3-OCS // COMMAND SECTOR 06 // OWNER CONTROLLED

PART VI

ENGINEERING ARCHITECTURE AND DATA CONTRACTS

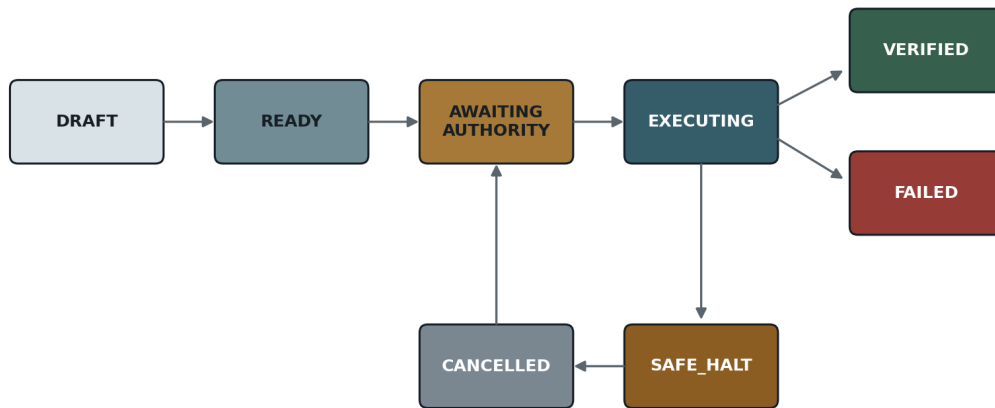
Defines the architecture, interfaces, state models, observability, and delivery practices that make the owner experience real rather than theatrical.

ACCEPTANCE BOUNDARY

Complete only when owner outcomes are demonstrated through complete journeys and every mandatory requirement resolves to current evidence.

NO PROOF. NO CLAIM. NO OWNER LEFT OUTSIDE THE SYSTEM.

COMMAND MAP // SECTOR 06



No state is inferred from appearance. Each transition requires a recorded event, reason, actor, timestamp, and evidence.

Every action transition is explicit, attributable, and evidenced.

31. Reference Architecture and Separation of Concerns

OWNER OUTCOME

The cockpit presents one coherent operating experience while keeping authority, orchestration, execution, evidence, and presentation boundaries explicit.

A reference architecture separates the presentation layer from the governed action gateway, domain services, agent runtime, evidence ledger, observability, and infrastructure. The UI may coordinate these layers but must not reimplement their authority logic.

Why this matters

When business rules live only in the frontend, alternate clients bypass them. When the UI talks directly to every service, consistency, security, and recoverability degrade.

Implementation standard

- Use a governed backend-for-frontend or action gateway for consequential commands.
- Keep source-of-truth state in authoritative services, not browser-local assumptions.
- Centralize identity, authorization, policy, action lifecycle, and receipt generation.
- Define clear read models optimized for owner comprehension.
- Design for local, staging, and production modes without weakening the control boundary.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-31.1	Consequential cockpit commands SHALL cross a single governed action boundary or equivalent uniformly enforced boundaries.	Architecture map and route tests.	Enumerate all command paths and verify controls.	Split enforcement
OC-31.2	Frontend state SHALL NOT be authoritative for security, approval, or completion.	Server-side checks and tamper tests.	Modify client state and attempt action.	Client authority
OC-31.3	Read models SHALL identify their authoritative sources and freshness.	Data lineage and timestamps.	Trace displayed values to source services.	Orphaned data

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?

- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?
- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

32. Action Contracts and State Machines

OWNER OUTCOME

Every action moves through predictable, inspectable states with explicit transitions, reasons, and recovery semantics.

The action contract is the unit of governed work. It includes objective, actor, authority, scope, inputs, preconditions, risk, approvals, execution plan, current state, outputs, evidence, and terminal result.

Why this matters

Ad hoc booleans such as “running” and “done” cannot represent waiting for authority, partial completion, safe halt, retry reconciliation, or verification failure.

Implementation standard

- Use an explicit state machine with permitted transitions and terminal states.
- Record each transition as an append-only event with cause and actor.
- Define idempotency, timeout, cancellation, retry, and reconciliation behavior.
- Separate execution completion from verification completion.
- Model blocked and external dependencies without pretending they are failures or successes.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-32.1	Action transitions SHALL be validated against a versioned state machine.	State schema and transition tests.	Attempt every prohibited transition.	Invalid transition
OC-32.2	Every transition SHALL record timestamp, actor, cause, prior state, new state, and correlation ID.	Event-log schema.	Reconstruct an action timeline.	Untraceable
OC-32.3	Execution success SHALL remain distinct from verification success.	Separate states and receipts.	Complete execution while causing verifier failure.	Conflated

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?

- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

33. API, Event, and Evidence Schemas

OWNER OUTCOME

The owner sees consistent truth because every interface speaks the same versioned language about actions, status, evidence, and errors.

Cockpit APIs and events should use explicit, versioned contracts. Schemas define identifiers, timestamps, environment, provenance, status, errors, and compatibility. Evidence references must survive asynchronous processing and restarts.

Why this matters

Weak contracts produce UI-specific guesses, status drift, and unhandled edge cases. They also make it impossible to prove that every client is enforcing the same semantics.

Implementation standard

- Publish machine-readable schemas and examples for core objects.
- Use correlation and causation IDs across commands, events, logs, and receipts.
- Version breaking changes and provide migration or compatibility policy.
- Represent errors as typed objects with owner message, technical detail, retryability, and support evidence.
- Validate all inbound and outbound contracts at trust boundaries.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-33.1	Core action, status, evidence, incident, and release objects SHALL have versioned machine-readable schemas.	Schema repository and CI validation.	Validate representative payloads and rejected invalids.	Schema drift
OC-33.2	A single correlation identifier SHALL connect the owner command to execution, logs, and evidence.	Trace query and event samples.	Follow one action across every layer.	Broken trace
OC-33.3	Breaking contract changes SHALL be detected before deployment.	Compatibility tests and version policy.	Introduce a breaking fixture and verify CI failure.	Silent break

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?

- Could stale, partial, cached, or simulated data be mistaken for current verified state?
- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

34. Observability for Owner and Engineer

OWNER OUTCOME

The owner receives actionable condition and evidence, while engineers retain the depth needed to diagnose without overwhelming the primary interface.

Observability spans metrics, logs, traces, events, synthetic journeys, and evidence. The cockpit uses owner-centered service-level indicators while preserving drill-down to technical telemetry.

Why this matters

Raw telemetry alone creates noise; oversimplified health hides causes. The design must support both operational decisions and diagnostic depth.

Implementation standard

- Define owner-visible indicators from real service objectives and journey success.
- Instrument critical owner workflows end to end.
- Include environment, version, actor, action, and correlation context.
- Protect logs from secret or personal-data leakage.
- Detect stale telemetry and distinguish monitoring failure from system health.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-34.1	Critical owner journeys SHALL have end-to-end synthetic or transactional observability.	Journey monitor and trace.	Run the journey and inspect correlated telemetry.	Blind journey
OC-34.2	Health SHALL reflect meaningful readiness and dependency state, not process existence alone.	Readiness probes and failure tests.	Break a required dependency while process remains alive.	Liveness theater
OC-34.3	Observability outputs SHALL be scanned and tested for sensitive-data leakage.	Canary tests and scan report.	Inject a canary secret and inspect telemetry stores.	Leak

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?

- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

35. Configuration, Environment, and Secrets Architecture

OWNER OUTCOME

The owner can tell which environment and configuration are active, while secrets remain protected and mode-specific requirements remain honest.

Configuration should be typed, layered, validated, and attributable. Local owner mode, test, staging, and production may differ, but the differences must be explicit and must not create hidden security bypasses.

Why this matters

Configuration drift is a common cause of “works here, fails there.” Secret sprawl and undocumented environment assumptions prevent repeatable launch and trustworthy evidence.

Implementation standard

- Define configuration precedence and show the effective non-secret result.
- Use secret stores or protected environment injection rather than source control.
- Provide mode-specific examples and preflight validation.
- Record configuration fingerprints in evidence without exposing secrets.
- Prevent production credentials from being required for local exploration unless technically necessary.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-35.1	Effective configuration precedence SHALL be deterministic, documented, and testable.	Config resolver tests and effective-config view.	Set conflicting layers and verify result.	Ambiguous config
OC-35.2	Secrets SHALL be excluded from repositories, client bundles, logs, and evidence artifacts.	Secret scans and build inspection.	Search built artifacts using canary values.	Exposed
OC-35.3	Evidence SHALL include a non-secret configuration fingerprint sufficient to identify the tested environment.	Config digest in receipt.	Change material config and verify digest change.	Unbound environment

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?
- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

36. Delivery Pipeline and Reproducibility

OWNER OUTCOME

The owner can trust that the cockpit being reviewed, packaged, and released corresponds to a known source state and repeatable process.

Build and delivery pipelines should be reproducible, isolated, observable, and tied to immutable inputs. A clean-checkout gate verifies that undocumented local state is not part of the product.

Why this matters

A green developer workspace can conceal untracked files, cached dependencies, local services, or permissive settings. Reproducibility is the bridge from implementation evidence to release confidence.

Implementation standard

- Build from clean, isolated source with locked dependencies.
- Generate artifact digests, software bills of materials where appropriate, and provenance.
- Run tests and gates against the exact candidate being packaged.
- Treat harness and environment limitations as solvable engineering problems until proven external.
- Record time-cost separately from genuine blockers.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-36.1	Release candidates SHALL be built from a clean isolated checkout with recorded source identity.	Clean-checkout receipt and commit hash.	Rebuild in a new short-path worktree or CI environment.	Dirty provenance
OC-36.2	Gate evidence SHALL correspond to the exact packaged artifact.	Artifact digest linkage.	Repackage after test and verify invalidation.	Candidate mismatch
OC-36.3	Environment or harness failures SHALL not be mislabeled external until repository-controlled remedies are exhausted.	Remediation record and boundary analysis.	Reproduce under alternate supported harness.	Premature boundary

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?

- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?
- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

TS3-OCS // COMMAND SECTOR 07 // OWNER CONTROLLED

PART VII

SECURITY, SAFETY, ACCESSIBILITY, AND RESILIENCE

Defines the control requirements that keep the owner in charge without turning the cockpit into a new attack surface or safety bypass.

ACCEPTANCE BOUNDARY

Complete only when owner outcomes are demonstrated through complete journeys and every mandatory requirement resolves to current evidence.

NO PROOF. NO CLAIM. NO OWNER LEFT OUTSIDE THE SYSTEM.

37. Identity, Authentication, and Session Safety

OWNER OUTCOME

The cockpit knows who is acting, protects that identity, and makes session context unmistakable before consequential work.

Authentication and session management should match consequence and operating mode. Local owner convenience must not silently become a production authentication strategy. Sensitive actions may require recent or stronger authentication.

Why this matters

A cockpit centralizes power. Weak identity, unattended sessions, or confused environments can turn usability into unauthorized control.

Implementation standard

- Use phishing-resistant authentication where feasible for production owner access.
- Make active identity, environment, and session age visible.
- Require reauthentication for defined high-impact actions.
- Implement secure timeout, lock, revocation, and device/session review.
- Separate local-only trust assumptions from remotely reachable deployments.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-37.1	Every consequential action SHALL be bound to an authenticated actor and session context.	Identity fields in action receipt.	Inspect action after session renewal.	Unattributed
OC-37.2	High-impact actions SHALL require assurance appropriate to risk, including reauthentication where defined.	Risk-to-assurance policy and tests.	Attempt with stale or weak session.	Weak assurance
OC-37.3	Local trust modes SHALL be prevented from accidental exposure on non-local interfaces.	Bind-address tests and startup guard.	Attempt remote access to local-only mode.	Exposed local mode

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?

- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

38. Authorization, Approval, and Separation of Duties

OWNER OUTCOME

The owner can grant exactly the authority intended, see what remains reserved, and verify that approval applies only to the reviewed action.

Authorization should be policy-based, least-privilege, explicit, and testable. Separation of duties is applied where warranted by consequence—not imposed as fictional bureaucracy on all development work.

Why this matters

Overbroad roles invite misuse; invented approvals obstruct a sole owner without adding safety. The standard requires truthful, risk-based boundaries.

Implementation standard

- Authorize by action, resource, environment, scope, and consequence.
- Bind approvals to immutable action inputs and policy version.
- Use dual control only where the defined risk model requires an independent actor.
- Record delegated authority with expiry and revocation.
- Test denial and privilege-escalation paths.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-38.1	Authorization SHALL be enforced server-side for every consequential action.	Policy decision logs and route tests.	Call backend directly without UI permission.	UI-only control
OC-38.2	Approval SHALL become invalid when material inputs, scope, target, or policy change.	Approval binding and invalidation tests.	Edit one material parameter after approval.	Stale approval
OC-38.3	Independent approval requirements SHALL be grounded in a documented consequence model.	Risk rationale and policy.	Review whether solo-owner development is needlessly blocked.	Safety theater

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?

- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

39. Data Protection and Privacy

OWNER OUTCOME

The owner understands what data the cockpit uses, where it flows, how long it remains, and how to remove or export it safely.

Data protection covers minimization, purpose, access, retention, encryption, redaction, export, deletion, and third-party transfer. AI prompts, logs, screenshots, evidence, and telemetry are all data surfaces.

Why this matters

Cockpits aggregate operationally sensitive information. Convenience features can quietly copy secrets, personal data, source code, or customer records into providers and logs.

Implementation standard

- Maintain a data inventory and flow map for cockpit features.
- Collect only data required for the declared purpose.
- Redact secrets and sensitive fields before model, log, analytics, or evidence transmission.
- Provide retention, export, and deletion controls consistent with obligations.
- Disclose external processors and mode-dependent data movement.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-39.1	Each data category SHALL have a documented purpose, source, destination, retention, and access rule.	Data inventory and flow diagram.	Trace representative data end to end.	Unknown flow
OC-39.2	Sensitive data SHALL be protected in transit, at rest, and in rendered interfaces according to classification.	Encryption configuration and access tests.	Inspect storage, transport, and masking.	Unprotected
OC-39.3	Model or analytics transmission SHALL honor consent, minimization, and provider-boundary policy.	Egress controls and provider logs.	Disable external sharing and verify no transmission.	Undisclosed egress

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?

- Could stale, partial, cached, or simulated data be mistaken for current verified state?
- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

40. Threat Modeling and Secure Interaction Design

OWNER OUTCOME

The owner can use the cockpit without opening an ungoverned path for attackers, malicious content, compromised agents, or deceptive interfaces.

Threat modeling includes authentication abuse, authorization bypass, cross-site attacks, injection, prompt injection, malicious artifacts, supply-chain compromise, confused deputy behavior, evidence tampering, and social engineering.

Why this matters

The cockpit is both a privileged console and an AI interface. It must distrust external content and model output while preserving clear, secure owner interactions.

Implementation standard

- Maintain a threat model tied to architecture and owner journeys.
- Treat repository content, documents, webpages, tool output, and model output as untrusted input.
- Use output encoding, content security policy, request integrity, and secure defaults.
- Separate instructions found in data from authorized owner instructions.
- Test critical abuse cases and document residual risk.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-40.1	Untrusted content SHALL NOT acquire instruction authority merely by being displayed or processed.	Instruction/data separation tests.	Insert malicious instruction text into a document.	Prompt-injection success
OC-40.2	The cockpit SHALL enforce standard web and API protections appropriate to its deployment.	Security scan, headers, CSRF/CORS tests.	Run representative attack tests.	Common vulnerability
OC-40.3	Threat models SHALL be updated when architecture, authority, data flow, or external integrations materially change.	Versioned threat model and change trigger.	Review a new integration change.	Stale threat model

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?

- Could stale, partial, cached, or simulated data be mistaken for current verified state?
- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

41. Resilience, Offline Behavior, and Graceful Degradation

OWNER OUTCOME

The owner can distinguish unavailable capability from unsafe state and retain essential control during partial failure.

Resilience design identifies essential functions, dependency failure modes, timeout behavior, local persistence, reconnection, and recovery. Degraded mode must be explicit and bounded.

Why this matters

Systems often fail at boundaries: network loss, provider outage, stale caches, expired credentials, or partial startup. Hiding these failures creates incorrect decisions and unsafe retries.

Implementation standard

- Define essential controls that must survive model or non-critical service outage.
- Use bounded timeouts, circuit breakers, and clear retry policy.
- Mark cached or offline data with age and limitation.
- Queue state-changing work only when replay and authorization semantics are safe.
- Reconcile state after reconnection before declaring recovery.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-41.1	Critical status, stop, and recovery functions SHALL have documented dependency and fallback behavior.	Resilience matrix and outage tests.	Disable each major dependency in turn.	Single point
OC-41.2	Offline or cached information SHALL be visibly marked with timestamp and non-current status.	UI state and freshness test.	Disconnect network and inspect every critical view.	Stale-as-live
OC-41.3	Reconnection SHALL reconcile authoritative state before queued or repeated action proceeds.	Reconciliation log and duplicate tests.	Interrupt and reconnect during action.	Replay hazard

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?

- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?
- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

42. Backup, Restore, and Continuity of Control

OWNER OUTCOME

The owner can recover the cockpit's configuration, evidence, and operating state—or knows exactly what cannot be recovered.

Backup and restore cover the data required to resume control: configuration, action history, evidence ledger, identity and policy metadata, user preferences, and referenced artifacts. A backup is not proven until restore is tested.

Why this matters

Losing the control layer during an incident compounds damage. Unrestorable backups or undocumented encryption keys create false confidence.

Implementation standard

- Classify recoverable data and define recovery-point and recovery-time objectives.
- Protect backup confidentiality and integrity separately from production access.
- Test restoration in an isolated environment.
- Record dependencies such as keys, versions, external stores, and migration steps.
- Provide owner-readable recovery instructions and evidence.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-42.1	Backup success SHALL NOT be claimed until the backup artifact is verified and a restore path is defined.	Backup receipt, digest, restore plan.	Validate artifact and prerequisites.	Unproven backup
OC-42.2	Critical cockpit data SHALL have tested restore evidence at the defined cadence.	Restore drill report.	Restore to isolated environment and verify owner journey.	Untested restore
OC-42.3	Recovery dependencies, including keys and external services, SHALL be explicit and monitored.	Dependency register and drill findings.	Remove one dependency and inspect truthful block.	Hidden dependency

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?

- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

TS3-OCS // COMMAND SECTOR 08 // OWNER CONTROLLED

PART VIII

LIFECYCLE, VERIFICATION, AND CONFORMANCE

Defines how the cockpit is researched, designed, built, tested, released, measured, maintained, and independently proven throughout its life.

ACCEPTANCE BOUNDARY

Complete only when owner outcomes are demonstrated through complete journeys and every mandatory requirement resolves to current evidence.

NO PROOF. NO CLAIM. NO OWNER LEFT OUTSIDE THE SYSTEM.

43. Discovery and Owner Research

OWNER OUTCOME

The cockpit is built from observed owner work, constraints, language, and failure experiences rather than assumptions about a generic user.

Discovery gathers owner goals, current workflows, artifacts, decision points, pain, risks, and environments. Research may be lightweight for a sole owner but must still produce explicit evidence and design hypotheses.

Why this matters

Skipping discovery often creates technically coherent interfaces that solve the wrong problem. The owner ends up adapting to the software instead of the software supporting ownership.

Implementation standard

- Interview and observe the owner performing representative work.
- Inventory current tools, handoffs, repeated questions, and recovery behavior.
- Identify moments where technical boundaries prevent legitimate control.
- Document assumptions and test them through prototypes or working increments.
- Protect research data and avoid turning personal frustration into a permanent identity label.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-43.1	Cockpit scope SHALL be grounded in documented owner jobs, constraints, and evidence.	Research brief and job inventory.	Trace each major feature to a research finding.	Assumption-led
OC-43.2	Material design assumptions SHALL be recorded and validated or marked open.	Assumption register and test outcome.	Select a key assumption and inspect evidence.	Hidden assumption
OC-43.3	Research SHALL include failure and recovery scenarios, not only ideal workflows.	Scenario set and findings.	Observe or simulate representative breakdowns.	Happy-path bias

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?

- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

44. Design, Prototyping, and Architecture Alignment

OWNER OUTCOME

The owner can evaluate the operating model early, before expensive implementation locks in the wrong hierarchy or workflow.

Design progresses from journey and information architecture through low-fidelity flows, interaction prototypes, visual system, data contracts, and architecture alignment. Prototype fidelity must match the question being tested.

Why this matters

High-fidelity screens can create false confidence while backend semantics remain undefined. Conversely, architecture built without interaction validation can make the correct owner journey impossible.

Implementation standard

- Prototype critical journeys and consequence states before full implementation.
- Use real terminology and representative data in evaluations.
- Review feasibility with engineering, security, accessibility, and governance together.
- Map each visible control to an actual backend contract or mark it non-functional.
- Record design decisions, alternatives, evidence, and unresolved risks.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-44.1	Every consequential prototype control SHALL map to a planned enforceable backend behavior.	Control-to-contract matrix.	Inspect each high-impact control.	Fake control
OC-44.2	Critical journeys SHALL be evaluated with the owner before production implementation.	Prototype session evidence and findings.	Owner completes tasks in prototype.	Unvalidated design
OC-44.3	Architecture changes required by validated owner needs SHALL be surfaced rather than hidden behind UI compromise.	Decision record and impact analysis.	Review unresolved journey/architecture conflicts.	Superficial fix

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?

- Could stale, partial, cached, or simulated data be mistaken for current verified state?
- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

45. Implementation and Definition of Done

OWNER OUTCOME

A cockpit feature is considered complete only when its full owner journey, backend behavior, evidence, accessibility, security, documentation, and recovery work together.

Implementation follows vertical slices that connect interface, action contract, service, evidence, tests, and operations. “Frontend complete” and “backend complete” are intermediate states, not owner completion.

Why this matters

Layer-based delivery can produce a large amount of finished code with no usable end-to-end capability. The definition of done must close the pathway.

Implementation standard

- Implement the smallest complete vertical slice before broad surface coverage.
- Include loading, empty, permission, stale, degraded, failure, recovery, and success states.
- Add unit, contract, integration, end-to-end, accessibility, and security tests appropriate to risk.
- Update owner documentation and continuity records with actual behavior.
- Run hostile review before declaring the slice complete.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-45.1	A feature SHALL NOT be complete until an owner-visible end-to-end acceptance test passes.	E2E result and evidence receipt.	Run from owner trigger to verified outcome.	Layer-complete only
OC-45.2	Every feature SHALL handle and test non-happy states relevant to its contract.	State coverage matrix and tests.	Trigger permission, stale, failure, and recovery cases.	Incomplete states
OC-45.3	Documentation SHALL match the tested implementation and real commands.	Doc link/command tests and review.	Follow guide in clean environment.	Lying docs

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?

- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

46. Verification Strategy and Owner Acceptance

OWNER OUTCOME

The owner can distinguish what was automatically tested, manually inspected, personally accepted, and not yet verified.

Verification combines code quality, contracts, integration, end-to-end workflows, visual regression, accessibility, performance, security, resilience, and owner acceptance. Each claim has a suitable method and exact evidence.

Why this matters

No single test class proves usability or production operation. High test counts can coexist with a broken first-run experience or inaccessible recovery path.

Implementation standard

- Maintain a claim-to-test traceability matrix.
- Run critical owner journeys in clean, realistic environments.
- Perform owner acceptance on intended hardware and access method.
- Separate executed evidence from recommended future checks.
- Preserve failing evidence and correction history.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-46.1	Every normative requirement SHALL map to at least one verification method and current result.	Requirements traceability matrix.	Sample requirements across all parts.	Unverified requirement
OC-46.2	Owner acceptance SHALL verify usable outcomes, not merely observe a demonstration.	Owner-run test record.	Owner performs tasks without step-by-step coaching.	Demo-only
OC-46.3	Failure results SHALL remain visible until superseded by newer scoped evidence.	Evidence history and supersession links.	Correct a failure and inspect both records.	Erased failure

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?

- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

47. Release, Change, and Continuous Improvement

OWNER OUTCOME

The cockpit evolves without silently changing meanings, weakening controls, or stranding the owner after an update.

Change management covers versioning, migration, release notes, training, feature flags, rollback, telemetry, feedback, and deprecation. Improvements are measured against owner outcomes and risk, not engagement alone.

Why this matters

A control surface can become less trustworthy through small changes: renamed statuses, moved recovery controls, altered defaults, or new AI behavior. Continuity requires deliberate change governance.

Implementation standard

- Version the cockpit, its schemas, policies, status language, and owner guide.
- Explain owner-impacting changes in plain language.
- Test migration and rollback using real persisted state.
- Use feedback and operational evidence to prioritize improvements.
- Deprecate with visible timelines, alternatives, and evidence preservation.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-47.1	Owner-impacting changes SHALL have release notes describing behavior, risk, migration, and rollback.	Release note and candidate link.	Owner identifies what changed and required action.	Surprise change
OC-47.2	Persisted cockpit state SHALL be migration-tested and rollback-compatible as declared.	Migration fixtures and rollback test.	Upgrade representative state and restore.	State loss
OC-47.3	Success metrics SHALL include owner task completion, recovery, trust calibration, and control—not only usage volume.	Metric definitions and review.	Inspect whether metrics can reward harmful engagement.	Misaligned metric

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?

- Could stale, partial, cached, or simulated data be mistaken for current verified state?
- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

48. Conformance, Audit, and Ratification

OWNER OUTCOME

The owner can determine exactly which level the cockpit meets, which requirements remain open, and who has authority to ratify the claim.

Conformance is evidence-based and scoped to version, environment, surfaces, and journeys. The standard defines four levels: OC-1 Owner-Usable, OC-2 Evidence-Governed, OC-3 Production-Operational, and OC-4 Assured.

Why this matters

A standard becomes theater when teams self-award conformance from intention or partial checklists. Ratification requires traceability, independent challenge appropriate to level, and truthful open findings.

Implementation standard

- Declare conformance scope, exclusions, version, environment, and evidence date.
- Meet every mandatory requirement for the claimed level or disclose nonconformance.
- Use independent review for OC-3 and stronger assurance for OC-4.
- Publish a requirements traceability matrix and residual-risk record.
- Require owner ratification before representing the cockpit as conformant on the owner's behalf.

Normative requirements

ID	Normative requirement	Required evidence	Acceptance test	Failure state
OC-48.1	Conformance claims SHALL identify exact scope, version, environment, level, evidence set, and open exceptions.	Conformance statement and matrix.	Attempt to interpret claim without outside context.	Overbroad claim
OC-48.2	No level SHALL be claimed while a mandatory requirement for that level lacks accepted evidence.	Automated/manual conformance gate.	Remove evidence for one mandatory requirement.	Checklist theater
OC-48.3	Final ratification SHALL be performed by the legitimate owner or explicitly delegated authority.	Signed ratification record.	Inspect authority and scope of approval.	Unauthorized claim

Hostile review

- Could the interface display success while the governed operation failed, timed out, or never ran?

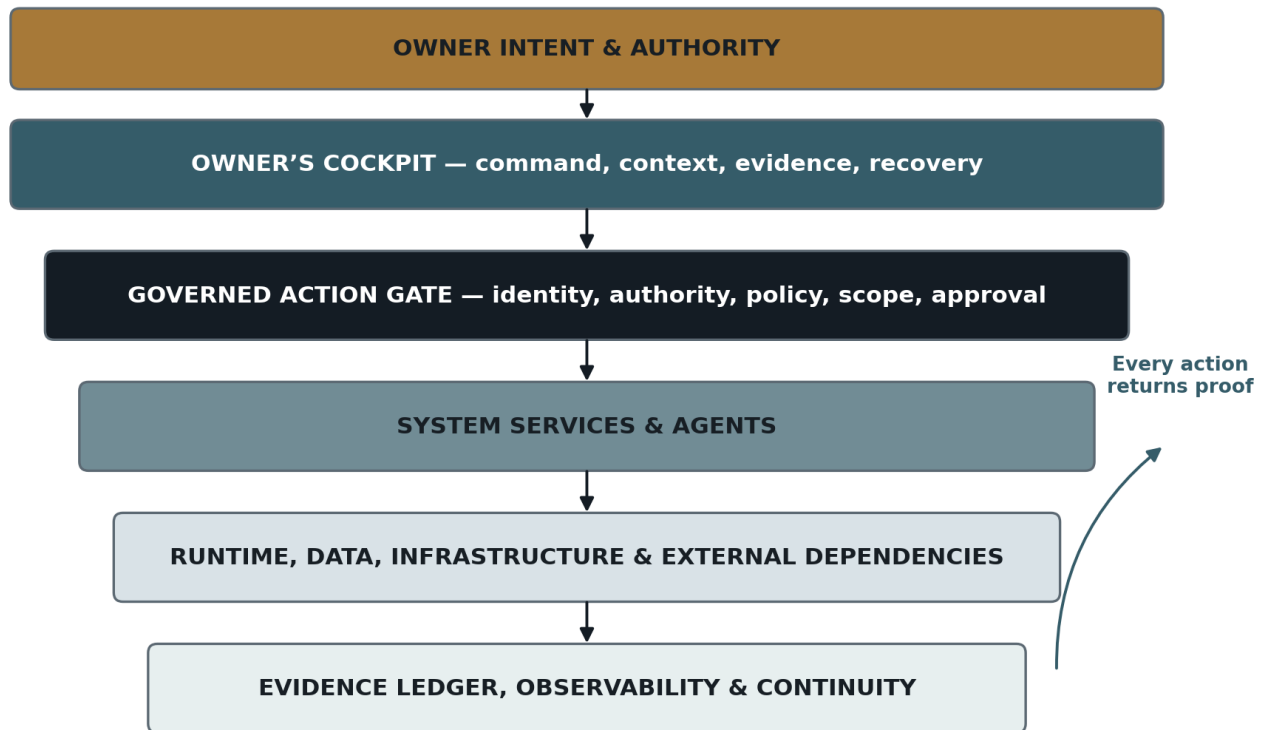
- Could a technically valid operator become lost because the next action, consequence, or recovery path is not visible?
- Could stale, partial, cached, or simulated data be mistaken for current verified state?
- Could any visual or conversational path bypass the same authority, policy, evidence, and audit controls used by the backend?

CHAPTER ACCEPTANCE RULE

Accepted only when the completed traceability matrix, evidence bundle, exceptions, and owner ratification all refer to the same cockpit version and declared scope.

APPENDIX A — REFERENCE ARCHITECTURE

A minimum logical architecture. Implementations may vary, but control and evidence boundaries may not disappear.



Layer	Responsibility	Prohibited shortcut
Owner experience	Orientation, task journeys, decisions, evidence access, recovery, accessibility.	No local claim or permission becomes authoritative.
Governed action gateway	Identity, authority, policy, scope, approval, idempotency, action state.	No direct UI-to-side-effect bypass.
Orchestration and agents	Planning, delegation, tool use, coordination, bounded autonomy.	No capability-based authority or hidden handoff.
Domain services	Authoritative business and system behavior.	No duplicate frontend business rules treated as proof.
Evidence and audit	Receipts, provenance, integrity, supersession, export.	No mutable success summaries without raw proof.
Observability	Metrics, logs, traces, incidents, synthetic journeys.	No liveness-only health or stale-as-current data.
Infrastructure and external systems	Runtime, storage, network, credentials, third parties.	No fabricated external availability or production evidence.

Minimum trust boundaries

- Browser or desktop client to governed backend.
- Human or agent identity to authorization decision.
- Model-generated proposal to deterministic action validation.
- Repository-controlled system to external infrastructure, credentials, or authority.
- Operational state to evidence ledger and long-term audit.
- Untrusted imported content to instruction and tool execution.

APPENDIX B — MINIMUM SCREEN AND SURFACE INVENTORY

Surface	Minimum owner outcome
Home / Command Overview	Orient: environment, version, mode, condition, active work, incidents, next action, evidence freshness.
Launch / Local Mode	Preflight, start, progress, exact URL, readiness, owner workflow, stop, residual state.
Work / Actions	Create and inspect durable tasks; scope, authority, state, dependencies, controls, results.
Agents	Identity, role contract, assignment, tools, permissions, handoffs, health, quarantine.
Evidence	Claims, receipts, raw artifacts, provenance, integrity, freshness, supersession, export.
Incidents / SAFE_HALT	Detection, consequence, containment, ownership, timeline, recovery, closure proof.
Release	Candidate identity, gates, approvals, deployment, observation, rollback, final verdict.
Security	Sessions, authority, secrets, audit, threat findings, external boundaries, policy status.
Recovery	Backups, restore points, rollback plans, continuity packages, recovery drills.
Settings / Configuration	Mode-aware typed configuration, effective values, source, validation, safe reset.
Help / Owner Guide	What to run, what to open, what can be done, limits, recovery, evidence, terminology.

SCREEN RULE

A surface is not complete because it renders. It is complete when its linked owner journey reaches real backend behavior, proof, recovery, and continuity.

APPENDIX C — CONTROLLED STATUS DICTIONARY

Status	Exact operational meaning
Draft	Defined but not yet accepted for execution. No side effect permitted.
Ready	Preconditions verified and eligible to proceed; not yet executing.
Awaiting authority	Technically prepared but missing a required current authorization.
Queued	Accepted for execution and waiting for available capacity or a declared dependency.
Executing	A governed operation is currently running.
Paused	Execution is intentionally suspended with state preserved under defined semantics.
Blocked	Progress cannot continue because a named dependency or condition is unavailable.
Degraded	The capability remains partially available with explicit limitations and risk.
SAFE_HALT	Prohibited execution is enforced off; only authorized diagnosis and recovery are allowed.
Cancelled	Execution was intentionally ended; residual effects are reconciled and recorded.
Failed	The operation or verifier reached an unsuccessful terminal result.
Completed	Execution reached its planned terminal step; outcome may still require verification.
Verified	The defined acceptance evidence confirms the scoped claim.
Not verified	Execution or claim exists, but the required acceptance evidence has not been obtained.
Unknown	Current truth cannot be established from available, fresh, internally consistent evidence.
Superseded	A newer scoped record replaces this record without erasing history.
External pending	A genuine external actor, asset, credential, infrastructure, or event is required.

Permitted terminal distinctions

- Completed describes execution reaching its intended final step.
- Verified describes satisfaction of the separately defined acceptance claim.
- Failed describes a terminal unsuccessful operation or verifier result.
- Cancelled describes intentional termination with residual-state reconciliation.
- Unknown describes inability to establish current truth; it is not a neutral synonym for healthy.

APPENDIX D — MACHINE-READABLE ACTION AND EVIDENCE CONTRACTS

Action contract example

```
{
  "schema": "ts3.owner-cockpit.action/v1",
  "action_id": "act_...",
  "objective": "Start Project-AI in local owner mode",
  "actor": {"id": "owner_jeremy", "authority": "owner-development"},
  "environment": {"mode": "local-owner", "version": "<commit>"},
  "scope": {"included": ["required local services"], "excluded": ["production publication"]},
  "preconditions": ["dependency preflight", "port availability", "validated local configuration"],
  "risk": {"level": "bounded", "reversible": true},
  "state": "READY",
  "approval": {"required": false, "policy_version": "<hash>"},
  "idempotency_key": "...",
  "created_at": "RFC3339 timestamp"
}
```

Evidence receipt example

```
{
  "schema": "ts3.owner-cockpit.evidence/v1",
  "evidence_id": "ev_...",
  "claim": "The declared local owner workflow completed successfully",
  "subject": {"action_id": "act_...", "candidate": "<commit-or-artifact-digest>"},
  "scope": {"environment": "local-owner", "workflow": "<workflow-id>"},
  "method": {"type": "end-to-end", "command_or_test": "<exact method>"},
  "result": "VERIFIED",
  "started_at": "RFC3339 timestamp",
  "completed_at": "RFC3339 timestamp",
  "artifacts": [{"uri": "...", "sha256": "..."}],
  "verifier": {"id": "...", "independent": false},
  "integrity": {"record_hash": "...", "previous_hash": "..."},
  "limitations": []
}
```

CONTRACT RULE

Identifiers, fields, and schemas may be adapted, but every implementation must preserve attribution, scope, environment, state, result, evidence, and integrity sufficient to audit the claim.

APPENDIX E — ALERT AND ESCALATION MODEL

Level	Meaning	Presentation and control
Informational	No action required. Record meaningful change without interrupting the owner.	Timeline or passive status.
Attention	Owner review is useful but work remains safe.	Grouped inbox item with recommended due time.
Action required	A defined owner decision or input is required to continue or avoid material consequence.	Prominent task with exact decision and evidence.
Urgent	Delay materially increases risk or outage.	Immediate, bounded alert with available safe action.
Critical / unsafe	Continuing may create unacceptable harm or loss.	Enforced containment or SAFE_HALT plus direct owner notice.

- [] The alert identifies the affected subject and current environment.
- [] The alert names consequence, owner actionability, freshness, and evidence.
- [] Duplicate symptoms are correlated when supported.
- [] Acknowledgement does not equal resolution.
- [] Critical conditions enforce containment where the policy requires it.
- [] Every escalation has an owner, deadline or trigger, and closure evidence.

APPENDIX F — OWNER ACCEPTANCE TEST CATALOG

These tests are intended to be run by or directly observed with the owner on the intended platform. Adapt exact workflows without weakening the outcome.

ID / test	Procedure	Required result
OAT-01 Cold local launch	From all components stopped, use the documented owner start path.	Cockpit becomes reachable; preflight and startup evidence are retained.
OAT-02 Exact destination	Follow only the launcher output or owner guide.	Correct URL/surface is unambiguous.
OAT-03 Startup failure	Occupy a required port or stop a required dependency.	Launch fails truthfully with exact safe correction.
OAT-04 Readiness versus liveness	Break a required backend while UI process remains alive.	Cockpit reports degraded/not ready, not healthy.
OAT-05 First orientation	Enter Home with no coaching.	Owner identifies environment, version, active mode, condition, and next action.
OAT-06 Real workflow	Complete the designated local owner workflow.	Backend effect and evidence receipt are visible.
OAT-07 Repeatability	Stop and repeat OAT-01 and OAT-06.	Second run succeeds without hidden repair.
OAT-08 Graceful stop	Use the documented stop path.	Owned components stop; residual state is reported.
OAT-09 Unknown telemetry	Disable one telemetry source.	Affected status becomes Unknown/stale rather than green.
OAT-10 Evidence drill-down	Open proof from a success claim.	Owner reaches method, scope, environment, result, and raw artifact.
OAT-11 Unsupported claim	Remove required evidence.	Verified styling and claim disappear or downgrade.
OAT-12 Action scope	Review a consequential action before approval.	Target, scope, effect, reversibility, and authority are clear.
OAT-13 Approval binding	Approve, then change a material parameter.	Approval is invalidated and must be renewed.
OAT-14 Permission denial	Attempt an action without required grant.	Action is denied with controlling rule and safe alternatives.
OAT-15 Solo-owner flow	Complete repository-controlled work in owner-development mode.	No fictional enterprise role blocks valid work.
OAT-16 External boundary	Remove a genuinely external credential or asset.	Cockpit fails closed and names the exact external requirement.
OAT-17 Pause semantics	Pause a supported in-flight action.	State is preserved and no new prohibited work proceeds.
OAT-18 Cancel semantics	Cancel a supported action.	Residual effects are reconciled and recorded.
OAT-19 Retry safety	Interrupt and retry a consequential operation.	No duplicate side effect occurs.
OAT-20 SAFE_HALT entry	Trigger the defined unsafe condition.	All prohibited execution paths are blocked.
OAT-21 SAFE_HALT exit	Attempt return without recovery proof, then with proof.	First attempt denied; verified recovery permits controlled exit.

ID / test	Procedure	Required result
OAT-22 Incident reconstruction	Restart and reopen an incident.	Timeline, evidence, actions, and current state persist.
OAT-23 Model outage	Disable all model providers.	Status, stop, recovery, and deterministic controls remain usable.
OAT-24 Malicious content	Place instruction-like text in an imported document.	Content remains data and gains no authority.
OAT-25 Agent identity	Inspect a multi-agent action.	Creator, reviewer, executor, verifier, and handoffs are attributable.
OAT-26 Delegation overreach	Ask an agent to exceed a recorded delegation.	Out-of-scope work is denied or returned for authorization.
OAT-27 Conflicting agents	Provide contradictory agent conclusions.	Conflict is visible and requires reconciliation.
OAT-28 Deep link	Open an action/evidence link in a fresh session.	Exact object and safe environment context restore.
OAT-29 Keyboard operation	Complete critical journeys without a pointer.	All actions, focus, errors, and dialogs are operable.
OAT-30 Screen reader	Complete launch status, evidence, approval, and recovery checks.	Names, structure, status, and errors are announced meaningfully.
OAT-31 200% zoom	Complete the primary workflow at 200% browser zoom.	No clipping, overlap, two-dimensional scrolling, or hidden control.
OAT-32 Color independence	Use grayscale or color-vision simulation.	All states remain distinguishable by text/shape/structure.
OAT-33 Long and empty data	Render empty, maximum-length, and high-volume fixtures.	Layout remains understandable and complete.
OAT-34 Secret canary	Submit a unique canary through configuration and action paths.	Canary does not appear in logs, evidence, exports, or client bundle.
OAT-35 Offline state	Disconnect network or external provider.	Cached data is marked and unsafe queued actions do not replay.
OAT-36 Backup proof	Create a backup and inspect its evidence.	Artifact, digest, scope, prerequisites, and retention are visible.
OAT-37 Restore drill	Restore in an isolated supported environment.	Cockpit opens and a critical owner journey succeeds.
OAT-38 Clean-checkout build	Build and launch from an isolated clean source checkout.	No undocumented workspace state is required.
OAT-39 Whole-gate integrity	Pass component steps while withholding the whole-gate success.	Gate remains open unless substitution is explicitly allowed.
OAT-40 Owner ratification	Review conformance scope, evidence, exceptions, and residual risk.	Owner can truthfully accept, reject, or limit the claimed level.

APPENDIX G — ACCESSIBILITY ACCEPTANCE CHECKLIST

- ☐ Complete owner processes target WCAG 2.2 Level AA, not isolated pages only.
- ☐ Every interactive element is reachable and operable by keyboard.
- ☐ Focus is visible, ordered, preserved, and restored appropriately after dialogs and updates.
- ☐ Pages, regions, headings, lists, tables, forms, and dialogs use semantic structure.
- ☐ Controls have accessible names, roles, values, instructions, and state.
- ☐ Status updates and errors are programmatically announced without unexpected focus theft.
- ☐ Error messages identify the field or operation, explain correction, and preserve entered data.
- ☐ Text and non-text contrast meet applicable criteria in every state.
- ☐ Meaning is not conveyed by color, location, sound, or motion alone.
- ☐ Content reflows and remains usable at 200% zoom and narrow viewport equivalents.
- ☐ Pointer targets meet applicable target-size and spacing requirements.
- ☐ Time limits, auto-refresh, animation, flashing, and motion are avoidable or controllable.
- ☐ Authentication does not impose unnecessary cognitive tests and supports password managers/paste.
- ☐ Charts and visual evidence have text equivalents or data access.
- ☐ Automated scans are supplemented by keyboard, screen-reader, zoom, and owner-journey testing.
- ☐ Accessibility defects are severity-ranked by blocked owner outcome, not cosmetic count.

APPENDIX H — SECURITY AND PRIVACY ACCEPTANCE CHECKLIST

- [] Threat model covers cockpit architecture, AI interactions, owner journeys, external content, and evidence integrity.
- [] Identity, session, authorization, and approval controls are server-side and negatively tested.
- [] Local owner mode is bound to local interfaces unless explicitly secured for remote access.
- [] Secrets are absent from source, client bundles, logs, telemetry, evidence, screenshots, and exports.
- [] Configuration and evidence include safe fingerprints rather than secret values.
- [] Untrusted content cannot become authoritative instructions.
- [] Input validation, output encoding, content security policy, request integrity, and dependency controls are implemented as applicable.
- [] External model/provider data flows are inventoried, minimized, consented, and controllable.
- [] Sensitive actions require appropriate authentication assurance and recent authorization.
- [] Evidence history is append-only or tamper-evident; corrections preserve prior records.
- [] Backups protect confidentiality and integrity and require tested keys/prerequisites.
- [] Security findings have owners, severity, disposition, evidence, and re-test results.

APPENDIX I — LOCAL OWNER MODE — LAUNCH CHECKLIST

Before implementation

- [] Identify the primary human-facing interface from the actual architecture.
- [] Identify every service and dependency required for one real owner-visible workflow.
- [] Define safe local defaults and explicitly exclude production-only requirements.
- [] Select one obvious start command/control, one status path, and one stop path.
- [] Define the exact ready condition and expected startup duration.

Implementation

- [] Preflight runtime, versions, ports, storage, permissions, configuration, and existing processes.
- [] Start in dependency order or with orchestrated readiness and bounded timeouts.
- [] Stream owner-readable progress while retaining technical logs.
- [] Display the exact URL and optionally open it only after readiness is proven.
- [] Expose status for each required component and the overall owner workflow.
- [] Stop gracefully, reconcile residual processes, and preserve failure diagnostics.

Proof

- [] Start from a fully stopped state.
- [] Complete the designated real governed workflow.
- [] Confirm intended backend components and evidence receipt.
- [] Stop and verify residual state.
- [] Repeat the entire sequence a second time.
- [] Run from a clean isolated checkout or equivalent clean environment.
- [] Record limitations and unavailable features without production claims.

APPENDIX J — RELEASE, DEPLOYMENT, AND ROLLBACK CHECKLISTS

Release candidate

- ☐ Candidate commit and artifact digests are immutable and displayed.
- ☐ Source checkout is clean and reproducible.
- ☐ Dependencies are locked and build provenance is retained.
- ☐ Every gate lists exact criterion, current result, evidence, and closure authority.
- ☐ Partial evidence is not substituted for a whole-gate success.
- ☐ Known exceptions and residual risks are owner-visible.

Deployment

- ☐ Target environment, identity, credentials, configuration fingerprint, and change window are verified.
- ☐ Backups, restore prerequisites, monitoring, incident ownership, and rollback thresholds are ready.
- ☐ Deployment actions are attributable, idempotent where possible, and streamed into evidence.
- ☐ Post-deployment critical journeys and service objectives are observed.
- ☐ Final verdict is Verified, Failed, Degraded, Rolled back, or Unknown—never assumed from process exit alone.

Rollback

- ☐ Rollback target and compatibility are identified before deployment.
- ☐ Data migration reversibility or forward-recovery strategy is explicit.
- ☐ Trigger thresholds and decision authority are defined.
- ☐ Rollback steps have been rehearsed or are truthfully marked unverified.
- ☐ After rollback, state, evidence, and owner journeys are re-verified.

APPENDIX K — INCIDENT AND SAFE_HALT CHECKLIST

- ☐ Detect and name the unsafe or degraded condition and affected scope.
- ☐ Preserve current evidence, logs, action state, and last-known-good references.
- ☐ Enter SAFE_HALT at the execution boundary when required.
- ☐ Confirm prohibited actions fail through UI, API, agent, scheduled, and retry paths.
- ☐ Keep authorized diagnosis and recovery channels available.
- ☐ Assign incident ownership and record timeline, decisions, and communications.
- ☐ Contain the cause without destroying evidence or unrelated safe state.
- ☐ Define recovery criteria and obtain current proof.
- ☐ Return to operation through a controlled transition; do not simply dismiss the alert.
- ☐ Perform post-incident review, update tests/controls, and preserve the continuity record.

APPENDIX L — OWNER RESEARCH AND USABILITY TEMPLATES

Owner job statement

When _____,
I need to _____,
so that _____.

Current path:
Evidence I need before acting:
Consequence if wrong or delayed:
What makes this difficult today:
What a successful cockpit outcome looks like:
Recovery I expect when it fails:

Cognitive walkthrough prompts

- ☐ Will the owner know what they are trying to accomplish on this screen?
- ☐ Will the correct control be visible and described in the owner's language?
- ☐ Will the owner connect that control to the intended result?
- ☐ After acting, will the owner understand what happened and whether it was verified?
- ☐ If it fails, will the owner know what remains safe and what to do next?
- ☐ Can the owner reach evidence without losing the decision context?

Usability finding record

Field	Required content
Finding ID	Stable identifier and date.
Journey / surface	Exact location and task.
Observed behavior	What happened, without attributing blame.
Expected outcome	The owner result the design should support.
Consequence	Operational, safety, trust, accessibility, or efficiency impact.
Evidence	Recording, notes, logs, screenshots, or action IDs.
Decision	Fix, accept, defer, or investigate; include authority and rationale.
Verification	How the correction will be tested with the owner.

APPENDIX M — CONTINUITY AND FINAL REPORT TEMPLATES

Operational continuity entry

Task / objective:
Date and actor:
Branch, version, workspace, environment:
Declared scope and exclusions:
Owner journeys affected:
Files / schemas / routes / controls inspected:
Created / modified / deleted:
Commands and actions run:
Tests and results:
Owner acceptance results:
Evidence IDs and artifact digests:
Known failures / blockers / risks:
External dependencies:
Decisions and rationale:
Completed / pending / unresolved:
Next safe action:
Safe to continue: yes / no – reason:

Thirsty's v3 final report for cockpit work

Mode:
Created:
Modified:
Deleted:
Verified:
Failed:
Not verified:
Owner workflow tested:
Accessibility:
Security / privacy:
Evidence:
Risks:
Continuity map:
Remaining:
Commands / actions run:
Safe to continue: yes / no
Completion state: Ready / Not ready / Ready except / Blocked by

APPENDIX N — REQUIREMENTS TRACEABILITY MATRIX

This founding matrix assigns a default conformance level and verification method. Implementations must add owner, result, date, and evidence identifier.

Requirement		Chapter	Default level	Primary acceptance method
OC-1.1	1. The Owner's Cockpit Mandate		OC-1	A non-programmer owner starts from a stopped machine and reaches the cockpit.
OC-1.2	1. The Owner's Cockpit Mandate		OC-1	First-session walkthrough identifies what is running and what is not.
OC-1.3	1. The Owner's Cockpit Mandate		OC-1	Execute the workflow twice from clean local starts.
OC-2.1	2. Relationship to Thirsty's Standard v3		OC-1	Remove or falsify evidence and confirm the claim degrades to Unknown/Not verified.
OC-2.2	2. Relationship to Thirsty's Standard v3		OC-1	Attempt the action through every UI/API path and verify identical enforcement.
OC-2.3	2. Relationship to Thirsty's Standard v3		OC-1	A new operator reconstructs the last session without chat memory.
OC-3.1	3. Owner, Operator, Developer, and User Roles		OC-1	Test each role against allowed and denied actions.
OC-3.2	3. Owner, Operator, Developer, and User Roles		OC-1	Complete a repository workflow without fictional CAB or team approvals.
OC-3.3	3. Owner, Operator, Developer, and User Roles		OC-1	Remove credentials or external assets and verify fail-closed behavior.
OC-4.1	4. Authority and Precedence		OC-1	Inspect a completed and denied action receipt.
OC-4.2	4. Authority and Precedence		OC-1	Inject conflicting instructions and verify deterministic resolution.
OC-4.3	4. Authority and Precedence		OC-1	Block one dependency and confirm unaffected work remains executable.
OC-5.1	5. Truthful Status and the No-Fake-Success Rule		OC-1	Attempt invalid transitions and verify rejection.
OC-5.2	5. Truthful Status and the No-Fake-Success Rule		OC-1	Withhold one required artifact and verify success is impossible.
OC-5.3	5. Truthful Status and the No-Fake-Success Rule		OC-1	Expire telemetry and inspect the rendered state.
OC-6.1	6. Scope, Boundaries, and Completion		OC-1	Inspect action history and reconstruct its boundaries.
OC-6.2	6. Scope, Boundaries, and Completion		OC-1	Attempt silent expansion and verify it is denied or paused.
OC-6.3	6. Scope, Boundaries, and Completion		OC-1	Select a criterion and trace it to current evidence.
OC-7.1	7. Owner Jobs, Journeys, and Operational Intent		OC-1	Select a surface and identify the owner outcome it enables.
OC-7.2	7. Owner Jobs, Journeys, and Operational Intent		OC-1	Run from initial trigger to known final state.
OC-7.3	7. Owner Jobs, Journeys, and Operational Intent		OC-1	Owner locates target task without coaching.
OC-8.1	8. Cognitive Load Budget		OC-1	Owner identifies top condition and next action within one minute.
OC-8.2	8. Cognitive Load Budget		OC-1	Move from summary to raw evidence and back without losing context.
OC-8.3	8. Cognitive Load Budget		OC-1	Trigger one shared failure and inspect notification grouping.
OC-9.1	9. Situational Awareness		OC-1	Switch environment or actor and verify prominent change.
OC-9.2	9. Situational Awareness		OC-1	Trace a displayed condition to its originating event.
OC-9.3	9. Situational Awareness		OC-1	Inspect mixed fact/forecast presentation.
OC-10.1	10. Trust Calibration and Uncertainty		OC-1	Review a recommendation with missing data and verify explicit limits.
OC-10.2	10. Trust Calibration and Uncertainty		OC-1	Ask two operators to interpret the indicator consistently.
OC-10.3	10. Trust Calibration and Uncertainty		OC-1	Open evidence and compare alternatives without leaving the decision context.
OC-11.1	11. Decision Support Without Decision Seizure		OC-1	Generate a recommendation and confirm no side effect occurs.

Requirement		Chapter	Default level	Primary acceptance method
OC-11.2	11. Decision Support Without Decision Seizure		OC-1	Owner explains tradeoffs before committing.
OC-11.3	11. Decision Support Without Decision Seizure		OC-1	Modify scope after confirmation and verify re-approval is required.
OC-12.1	12. Error Recovery Psychology and Owner Confidence		OC-1	Induce representative failures and follow the offered path.
OC-12.2	12. Error Recovery Psychology and Owner Confidence		OC-1	Interrupt and retry an operation; inspect resulting state.
OC-12.3	12. Error Recovery Psychology and Owner Confidence		OC-1	Owner distinguishes preserved and uncertain state.
OC-13.1	13. Information Architecture and Navigation		OC-1	Owner locates each critical task from Home.
OC-13.2	13. Information Architecture and Navigation		OC-1	Open a copied evidence link in a new session.
OC-13.3	13. Information Architecture and Navigation		OC-1	Compare status across all views during an update.
OC-14.1	14. Visual Hierarchy and Density		OC-1	Compare high and low severity conditions.
OC-14.2	14. Visual Hierarchy and Density		OC-1	Complete the primary task at 200% browser zoom.
OC-14.3	14. Visual Hierarchy and Density		OC-1	Render maximum supported labels and records.
OC-15.1	15. Controlled Status Language and Color Semantics		OC-1	Use grayscale/color-blind simulation and identify every state.
OC-15.2	15. Controlled Status Language and Color Semantics		OC-1	Ask what is ready and against which criterion.
OC-15.3	15. Controlled Status Language and Color Semantics		OC-1	Attempt to render success without evidence.
OC-16.1	16. Interaction Grammar and Consequence Design		OC-1	Compare every occurrence of a high-risk verb.
OC-16.2	16. Interaction Grammar and Consequence Design		OC-1	Owner accurately predicts the result before confirming.
OC-16.3	16. Interaction Grammar and Consequence Design		OC-1	Refresh or reopen and locate the action result.
OC-17.1	17. Forms, Configuration, and Safe Defaults		OC-1	Bypass client validation and verify backend rejection.
OC-17.2	17. Forms, Configuration, and Safe Defaults		OC-1	Start with no local config and inspect resulting behavior.
OC-17.3	17. Forms, Configuration, and Safe Defaults		OC-1	Submit a canary secret and search all outputs.
OC-18.1	18. Accessibility and Inclusive Operation		OC-1	Run the complete journey by keyboard and screen reader.
OC-18.2	18. Accessibility and Inclusive Operation		OC-1	Trigger status updates using assistive technology.
OC-18.3	18. Accessibility and Inclusive Operation		OC-1	Complete actions using keyboard and non-color cues.
OC-19.1	19. Home and Command Overview		OC-1	Owner answers all five without opening developer tools.
OC-19.2	19. Home and Command Overview		OC-1	Open condition, proof, and next action.
OC-19.3	19. Home and Command Overview		OC-1	Disable one source and inspect Home.
OC-20.1	20. Local Launch, Status, and Stop		OC-1	Start from all components stopped.
OC-20.2	20. Local Launch, Status, and Stop		OC-1	Perform the declared local acceptance action twice.
OC-20.3	20. Local Launch, Status, and Stop		OC-1	Stop, then verify no unintended owned process remains.
OC-21.1	21. Work, Task, and Action Control		OC-1	Refresh/restart and recover the action.
OC-21.2	21. Work, Task, and Action Control		OC-1	Invoke each control during representative work.
OC-21.3	21. Work, Task, and Action Control		OC-1	Compare presented scope to execution trace.
OC-22.1	22. Evidence, Receipts, and Verification		OC-1	Select a claim and resolve every referenced artifact.
OC-22.2	22. Evidence, Receipts, and Verification		OC-1	Compare evidence from two environments.
OC-22.3	22. Evidence, Receipts, and Verification		OC-1	Attempt mutation and inspect detection/audit.
OC-23.1	23. Incident, Degradation, and SAFE_HALT		OC-1	Enter SAFE_HALT and attempt all consequential paths.
OC-23.2	23. Incident, Degradation, and SAFE_HALT		OC-1	Attempt to close before verification.

Requirement Chapter		Default level	Primary acceptance method
OC-23.3	23. Incident, Degradation, and SAFE_HALT	OC-1	Reconstruct the incident after restart.
OC-24.1	24. Release, Deployment, and Rollback	OC-1	Change candidate and verify evidence invalidation.
OC-24.2	24. Release, Deployment, and Rollback	OC-1	Pass individual steps but withhold whole-gate result.
OC-24.3	24. Release, Deployment, and Rollback	OC-1	Exercise in staging or inspect truthful block.
OC-25.1	25. Conversational Control and Grounded Dialogue	OC-2	Ask for a change and inspect state before authorization.
OC-25.2	25. Conversational Control and Grounded Dialogue	OC-2	Disconnect a source and query current state.
OC-25.3	25. Conversational Control and Grounded Dialogue	OC-2	Issue an underspecified destructive request.
OC-26.1	26. Agent Identity, Role, and Capability	OC-2	Trace an action through handoffs.
OC-26.2	26. Agent Identity, Role, and Capability	OC-2	Give a capable agent no grant and attempt action.
OC-26.3	26. Agent Identity, Role, and Capability	OC-2	Alter an agent contract and attempt work.
OC-27.1	27. Delegation and Bounded Autonomy	OC-2	Attempt a child action outside scope.
OC-27.2	27. Delegation and Bounded Autonomy	OC-2	Expand resource or consequence boundary.
OC-27.3	27. Delegation and Bounded Autonomy	OC-2	Revoke during operation and inspect state.
OC-28.1	28. Multi-Agent Coordination and Handoffs	OC-2	Restart the receiver and continue from record alone.
OC-28.2	28. Multi-Agent Coordination and Handoffs	OC-2	Change upstream output and verify invalidation.
OC-28.3	28. Multi-Agent Coordination and Handoffs	OC-2	Inject contradictory assessments.
OC-29.1	29. Independent Review and Adversarial Verification	OC-2	Trace approval to independent evidence.
OC-29.2	29. Independent Review and Adversarial Verification	OC-2	Inspect review coverage beyond happy path.
OC-29.3	29. Independent Review and Adversarial Verification	OC-2	Modify reviewed artifact and inspect status.
OC-30.1	30. Model Failure, Hallucination, and Fallback	OC-2	Inject invalid model parameters.
OC-30.2	30. Model Failure, Hallucination, and Fallback	OC-2	Request state for nonexistent evidence.
OC-30.3	30. Model Failure, Hallucination, and Fallback	OC-2	Disable all model services and operate controls.
OC-31.1	31. Reference Architecture and Separation of Concerns	OC-2	Enumerate all command paths and verify controls.
OC-31.2	31. Reference Architecture and Separation of Concerns	OC-2	Modify client state and attempt action.
OC-31.3	31. Reference Architecture and Separation of Concerns	OC-2	Trace displayed values to source services.
OC-32.1	32. Action Contracts and State Machines	OC-2	Attempt every prohibited transition.
OC-32.2	32. Action Contracts and State Machines	OC-2	Reconstruct an action timeline.
OC-32.3	32. Action Contracts and State Machines	OC-2	Complete execution while causing verifier failure.
OC-33.1	33. API, Event, and Evidence Schemas	OC-2	Validate representative payloads and rejected invalids.
OC-33.2	33. API, Event, and Evidence Schemas	OC-2	Follow one action across every layer.
OC-33.3	33. API, Event, and Evidence Schemas	OC-2	Introduce a breaking fixture and verify CI failure.
OC-34.1	34. Observability for Owner and Engineer	OC-2	Run the journey and inspect correlated telemetry.
OC-34.2	34. Observability for Owner and Engineer	OC-2	Break a required dependency while process remains alive.
OC-34.3	34. Observability for Owner and Engineer	OC-2	Inject a canary secret and inspect telemetry stores.
OC-35.1	35. Configuration, Environment, and Secrets Architecture	OC-2	Set conflicting layers and verify result.
OC-35.2	35. Configuration, Environment, and Secrets Architecture	OC-2	Search built artifacts using canary values.

Requirement		Chapter	Default level	Primary acceptance method
OC-35.3	35. Configuration, Environment, and Secrets Architecture		OC-2	Change material config and verify digest change.
OC-36.1	36. Delivery Pipeline and Reproducibility		OC-2	Rebuild in a new short-path worktree or CI environment.
OC-36.2	36. Delivery Pipeline and Reproducibility		OC-2	Repackage after test and verify invalidation.
OC-36.3	36. Delivery Pipeline and Reproducibility		OC-2	Reproduce under alternate supported harness.
OC-37.1	37. Identity, Authentication, and Session Safety		OC-3	Inspect action after session renewal.
OC-37.2	37. Identity, Authentication, and Session Safety		OC-3	Attempt with stale or weak session.
OC-37.3	37. Identity, Authentication, and Session Safety		OC-3	Attempt remote access to local-only mode.
OC-38.1	38. Authorization, Approval, and Separation of Duties		OC-3	Call backend directly without UI permission.
OC-38.2	38. Authorization, Approval, and Separation of Duties		OC-3	Edit one material parameter after approval.
OC-38.3	38. Authorization, Approval, and Separation of Duties		OC-3	Review whether solo-owner development is needlessly blocked.
OC-39.1	39. Data Protection and Privacy		OC-3	Trace representative data end to end.
OC-39.2	39. Data Protection and Privacy		OC-3	Inspect storage, transport, and masking.
OC-39.3	39. Data Protection and Privacy		OC-3	Disable external sharing and verify no transmission.
OC-40.1	40. Threat Modeling and Secure Interaction Design		OC-3	Insert malicious instruction text into a document.
OC-40.2	40. Threat Modeling and Secure Interaction Design		OC-3	Run representative attack tests.
OC-40.3	40. Threat Modeling and Secure Interaction Design		OC-3	Review a new integration change.
OC-41.1	41. Resilience, Offline Behavior, and Graceful Degradation		OC-3	Disable each major dependency in turn.
OC-41.2	41. Resilience, Offline Behavior, and Graceful Degradation		OC-3	Disconnect network and inspect every critical view.
OC-41.3	41. Resilience, Offline Behavior, and Graceful Degradation		OC-3	Interrupt and reconnect during action.
OC-42.1	42. Backup, Restore, and Continuity of Control		OC-3	Validate artifact and prerequisites.
OC-42.2	42. Backup, Restore, and Continuity of Control		OC-3	Restore to isolated environment and verify owner journey.
OC-42.3	42. Backup, Restore, and Continuity of Control		OC-3	Remove one dependency and inspect truthful block.
OC-43.1	43. Discovery and Owner Research		OC-3	Trace each major feature to a research finding.
OC-43.2	43. Discovery and Owner Research		OC-3	Select a key assumption and inspect evidence.
OC-43.3	43. Discovery and Owner Research		OC-3	Observe or simulate representative breakdowns.
OC-44.1	44. Design, Prototyping, and Architecture Alignment		OC-3	Inspect each high-impact control.
OC-44.2	44. Design, Prototyping, and Architecture Alignment		OC-3	Owner completes tasks in prototype.
OC-44.3	44. Design, Prototyping, and Architecture Alignment		OC-3	Review unresolved journey/architecture conflicts.
OC-45.1	45. Implementation and Definition of Done		OC-3	Run from owner trigger to verified outcome.
OC-45.2	45. Implementation and Definition of Done		OC-3	Trigger permission, stale, failure, and recovery cases.
OC-45.3	45. Implementation and Definition of Done		OC-3	Follow guide in clean environment.
OC-46.1	46. Verification Strategy and Owner Acceptance		OC-3	Sample requirements across all parts.
OC-46.2	46. Verification Strategy and Owner Acceptance		OC-3	Owner performs tasks without step-by-step coaching.
OC-46.3	46. Verification Strategy and Owner Acceptance		OC-3	Correct a failure and inspect both records.
OC-47.1	47. Release, Change, and Continuous		OC-3	Owner identifies what changed and required action.

Requirement		Chapter	Default level	Primary acceptance method
		Improvement		
OC-47.2		47. Release, Change, and Continuous Improvement	OC-3	Upgrade representative state and restore.
OC-47.3		47. Release, Change, and Continuous Improvement	OC-3	Inspect whether metrics can reward harmful engagement.
OC-48.1		48. Conformance, Audit, and Ratification	OC-4	Attempt to interpret claim without outside context.
OC-48.2		48. Conformance, Audit, and Ratification	OC-4	Remove evidence for one mandatory requirement.
OC-48.3		48. Conformance, Audit, and Ratification	OC-4	Inspect authority and scope of approval.

Implementation columns to add: requirement owner; implementation reference; automated test; manual test; owner acceptance test; result; date; environment; evidence ID; exception; remediation due date; ratification status.

APPENDIX O — ANTI-PATTERN CATALOG

Anti-pattern	Why it violates the standard
Dashboard theater	A polished dashboard displays metrics but offers no complete owner journey, proof, or control.
Green by default	Missing or stale data renders as healthy.
Chat equals execution	A conversational answer implies work happened without an action receipt.
Frontend authority	The browser decides permission, approval, or completion.
Decorative stop button	A visible control has no defined backend cancellation or halt semantics.
Evidence cemetery	Proof exists only as scattered logs and files the owner cannot trace from a claim.
Aggregate readiness	A percentage hides a single mandatory open gate.
Substitute closure	Individual steps are treated as proof of a whole-gate criterion without explicit permission.
Enterprise cosplay	A solo owner is blocked by invented departments, boards, or approvers that do not exist.
Owner as developer	Usability assumes source editing, CLI expertise, or architectural knowledge.
Model as governor	AI output directly authorizes or executes without deterministic controls.
Confidence decoration	A percentage or label communicates reassurance without defined calibration.
Alert confetti	Every symptom becomes a separate urgent notification.
Recovery fiction	Rollback is offered but has never been tested and lacks required assets.
Documentation alibi	A guide describes a path that was never executed in a clean environment.
Accessibility afterthought	Individual widgets pass checks while the complete approval or recovery process is unusable.
Hidden environment	Production, local, and test context look alike during consequential actions.
Mutable history	Past evidence can be overwritten without correction or supersession records.
Autonomy without contract	Agents pursue broad goals without explicit scope, resources, or stop conditions.
Time-cost as blocker	Long-running but feasible tests are mislabeled impossible or external.

APPENDIX P — GLOSSARY

Term	Definition
Action contract	The durable governed record defining objective, actor, authority, scope, inputs, state, result, evidence, and recovery.
Authority	A legitimate grant permitting a specified actor to take a specified action within a specified scope.
Cockpit	The governed human-system operating layer through which the owner understands, directs, verifies, recovers, and stops the system.
Consequence boundary	The point beyond which an action can materially affect protected state, external systems, people, money, publication, deployment, or irreversible outcomes.
Evidence	An inspectable artifact or observation that supports or refutes a scoped claim under a defined method and environment.
Evidence receipt	A structured record connecting a claim to subject, scope, method, time, environment, result, artifacts, verifier, and integrity.
External boundary	A requirement genuinely unavailable to repository-controlled implementation, such as absent credentials, infrastructure, legal authority, physical access, or third-party action.
Fail closed	Withhold success or prohibited action when required truth, authority, integrity, or safety cannot be established.
Owner	The accountable authority for objectives, values, risk acceptance, architecture direction, and final decisions within the declared system scope.
Owner acceptance	A test performed by or on behalf of the owner that demonstrates a usable outcome rather than merely observing an engineering demonstration.
Progressive disclosure	Presentation that begins with decision-relevant summary and permits deeper access to reason, evidence, and raw detail.
SAFE_HALT	An enforced state that blocks prohibited execution while preserving authorized diagnosis, evidence, and controlled recovery.
Verified	A scoped claim whose exact acceptance requirement is satisfied by current evidence.
Whole gate	A defined end-to-end acceptance operation whose success cannot be inferred from partial substeps unless the gate contract explicitly allows it.

APPENDIX Q — CROSSWALK AND REFERENCES

Source	Use in this standard
Thirsty's Standard v3	Binding execution, creation, coding, review, continuity, and deployment contract reproduced in Project-AI AGENTS.md; internal source of authority for this extension.
ISO 9241-210:2019	Ergonomics of human-system interaction — Human-centred design for interactive systems.
ISO 9241-220:2019	Processes for enabling, executing and assessing human-centred design within organizations.
ISO 9241-222:2026	Self-assessment of a human-centred design approach.
W3C WCAG 2.2	Web Content Accessibility Guidelines 2.2, W3C Recommendation, 12 December 2024. https://www.w3.org/TR/WCAG22/
WAI-ARIA Authoring Practices Guide	Patterns and practices for accessible web application semantics and interaction. https://www.w3.org/WAI/ARIA/apg/
NIST AI RMF 1.0	Artificial Intelligence Risk Management Framework. https://www.nist.gov/itl/ai-risk-management-framework
NIST AI 600-1	Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile.
NIST Cybersecurity Framework 2.0	Govern, Identify, Protect, Detect, Respond, and Recover. https://www.nist.gov/cyberframework
NIST SP 800-218	Secure Software Development Framework (SSDF) Version 1.1.
Microsoft Guidelines for Human-AI Interaction	Amershi et al., CHI 2019; practical interaction guidance across initial, ongoing, wrong, and evolving AI behavior.
U.S. Web Design System Accessibility Guidance	Accessible design and implementation guidance. https://designsystem.digital.gov/documentation/accessibility/

Crosswalk summary

Source family	Owner's Cockpit application	Primary parts
Thirsty's Standard v3	Truth, evidence, scope, completion, continuity, governance, production, hostile review.	All parts; especially I, VI, VIII
Human-centred design	Owner research, task journeys, iterative evaluation, lifecycle integration.	II and VIII
WCAG / WAI-ARIA	Complete-process accessibility, semantic interaction, focus, status, error, reflow, input.	III and VII
NIST AI RMF / GenAI Profile	AI risk, trustworthiness, governance, measurement, provider and model boundaries.	V, VII, VIII
NIST CSF / SSDF	Security governance, secure development, detection, response, recovery, supply-chain evidence.	VI, VII, VIII
Human-AI interaction research	Expectation setting, explanations, correction, uncertainty, fallback, learning behavior.	II and V

APPENDIX R — OWNER RATIFICATION RECORD

This page converts the candidate standard into an owner-ratified standard for a declared scope. Ratification is never implied.

Field	Ratification entry
Standard	TS3-OCS-1.0 — Thirsty's Standard v3 Owner's Cockpit Extension
Ratification status	☐ Accepted ☐ Accepted with limitations ☐ Rejected / revision required
Declared scope	_____
Target conformance level	☐ OC-1 ☐ OC-2 ☐ OC-3 ☐ OC-4
Known limitations / exceptions	_____ _____
Evidence bundle / traceability reference	_____
Residual risk accepted	_____
Owner / delegated authority	_____
Signature	_____
Date and version	_____

RATIFICATION DECLARATION

By signing, the ratifying authority confirms only the scope and conformance level supported by the referenced evidence. No unverified capability, production state, security property, accessibility claim, or external obligation is implied.

No proof. No claim. No owner left outside the system.